

SmartStream TLM Reconciliations Premium 3.x Interview Guide

Prepared for a Singapore banking SmartStream TLM Consultant / Techno-Functional Consultant interview at approximately 5+ year consultant level.

IMPORTANT ACCURACY NOTE

This guide is intentionally interview-oriented but conservative. It separates confirmed/publicly documented SmartStream TLM Reconciliations Premium concepts from client/version-specific implementation details. It does not invent physical backend table names, MI syntax, ServiceMix filenames, proprietary commands, queue table names, configuration filenames or proprietary APIs. Where a bank asks for exact table names or filenames, the defensible 5+ year answer is: Version/client implementation dependent - verify against the client's schema/configuration; then explain the logical area you would inspect.

Research basis and public sources consulted

- SmartStream/Celent custom report, Reconciliation Systems: Solutions for Capital Markets Firms, 2024. Confirms TLM Reconciliations Premium coverage, MI loading capabilities, matching modes, TLM View, TLM Control, exception lifecycle, technology/deployment and database support: <https://smart.stream/wp-content/uploads/smartstream-resources/2024-05-22-Industry-Report-Celent-Custom-Report-Reconciliation-Systems.pdf>
- SmartStream TLM Reconciliations Premium executive summary. Confirms services: SmartRecs, TLM View, TLM Matching, TLM Exception Management, TLM Proofing, TLM Message Integration, TLM Persist, TLM Archive, TLM Control and TLM SmartSchema; also confirms flexible data loading, ETL, enrichment and persistence options: https://gd-celent-prod-cms.celent.com/v1/downloader/980651284_eeaa438c41.pdf
- Manpower Singapore SmartStream TLM Consultant JD. Confirms Singapore role requirements: TLM 3.x, TLM Control, Smart Studio, Recon Admin, RDBMS, Windows/Unix/Linux, ETL, testing and stakeholder management: <https://www.manpower.com.sg/jobs/information-and-communications-technology/smartstream-tlm-consultant/159829>
- S.i. Systems SmartStream TLM RP 3.0.9 consultant JD. Confirms TLM RP 3.0.9 implementation, TLM Control, SQL, integration and workflow support expectations: <https://www.sisystems.com/jobs/senior-smartstream-tlm-consultant-to-implement-configure-and-optimize-tlm-reconciliation-premium-tlm-rp-3-0-9-for-a-financial-reconciliation-project/330030003200340031003200/>
- Euronext ISO 15022 message specifications. Confirms securities reconciliation messages MT535, MT536, MT537 and MT548 purpose in an ISO 15022 context: <https://www.euronext.com/sites/default/files/2024-02/Layouts%20ISO%2015022%20Messages-Specifications.pdf>
- Clearstream Xact via Swift user guide. Confirms securities statements and corporate action message families in a custody context: <https://www.clearstream.com/caas/v1/media/1288716/data/4406514caed9a378b785423967e888da/swift-ug.pdf>
- HANDD/Tectia overview, sourced from vendor material. Confirms Tectia as SSH/SFTP/SCP secure transfer technology, interoperability and supported OS families: <https://support.handd.co.uk/hc/en-gb/articles/34373966855953-About-Tectia-SSH-Client-Server>
- Apache ServiceMix overview by Savoir Technologies. Confirms ServiceMix as an open source ESB/integration container using Karaf, ActiveMQ, Camel, CXF and OSGi concepts: <https://www.savoirtech.com/servicemix>

1. TLM architecture

Confirmed/public TLM RP picture: TLM Reconciliations Premium is a reconciliation-agnostic platform with services around loading/integration, matching, exception management, proofing, workflow/control, persistence/archive, dashboards and SmartSchema metadata. Public material describes TLM Message Integration/MI for data loading, TLM Matching for matching, TLM Control for no-code BPM/workflow customisation, TLM View for dashboards and operational UI, TLM Exception Management for break lifecycle, TLM Proofing for mathematical proofs, TLM Persist/Archive for persistence and archival choices, and SmartSchema as the metadata layer over the database structures.

Version note: the interview JD is TLM 3.x. Public material confirms TLM RP 3.x roles exist, including TLM RP 3.0.9 in a recent job description. Exact screens, database structures, service names and queue implementation vary by client build, patch level and deployment model.

Layer | Main responsibility | TLM component/logical area | Failure symptom | How to diagnose
 Input/connectivity | Receive file/message/database extract | Tectia/SFTP, middleware, scheduler, ServiceMix/client integration | File missing, late or wrong name | Check transfer logs, landing directory, file size, permissions, scheduler status
 Mapping/loading | Parse, validate, transform, enrich, reject or accept | TLM MI / DMW / ETL / mapping layer | Load failed, zero records, partial rejects | Check load control, reject files/tables, MI/ETL logs, record counts
 Business data | Store canonical transaction/item/balance/static views | BDR / SmartSchema / persistence logical areas | Data loaded but invisible or wrong fields | Trace source transaction ID, file ID, feed ID, account, currency, amount, date
 Static/model | Define sets, recons, data sources, accounts, workflow, rules | Recon Admin, SmartStudio/Smart Studio, TLM Control | Wrong recon/account/queue/matching | Compare static setup to source mapping and requirement spec
 Matching | Scope, group/populate, compare, pass quality, propose/create match | TLM Matching service | Items remain unmatched, no proposals, wrong proposals | Check triggering, workflow queue, matching logs, result/status areas
 Exception/workflow | Route breaks, assign, investigate, resolve, close | TLM Control, Exception Management, queues/inboxes/workfolders | Breaks not assigned, queue stuck, stale exceptions | Check workflow state, queue age, service status, roles, audit trail
 User/reporting | Operational dashboards, MI/control reports | TLM View/WebConnect/reporting | Records not visible to user | Check data status, dashboard filter, permissions, user role, cached view metadata

Interview Question

Explain the TLM 3.x architecture end to end.

20-second answer

TLM receives data through files/messages or extracts, maps and loads it through MI/ETL into the business data layer, associates it to static recon setup, initiates workflow, runs matching, creates proposals/matches/breaks, and exposes results in TLM View.

1-minute answer

I describe it in layers: connectivity, mapping/loading, BDR/business data, static configuration, workflow/control, matching, exceptions and dashboards. TLM Control controls workflow, TLM Matching performs the matching, MI/ETL maps and validates inbound data, and TLM View is the operational UI.

Senior 5+ year answer

At senior level I also separate data-load success from reconciliation success. A file can arrive and load, but matching may not run if proposal triggering, required input availability, workflow initiation or matching service is not ready. I would prove each layer with counts, status, logs and UI evidence, not by guessing a table name.

Real project example

For a cash recon, MT940/statement and ledger extracts land through SFTP, DMW/MI maps account, currency, amount, value date and reference, TLM creates items under the right account/recon, matching groups statement and ledger items, then exceptions route to an operations inbox in TLM View.

Technical deep dive

Important public confirmation: SmartStream public material describes MI loading with enrichment/transformation/lookups, matching immediately/scheduled/manual, TLM View dashboards, TLM Control workflow and exception management. Physical backend implementation is client/version dependent.

SQL/backend investigation

Start from <TLM_LOAD_BATCH_TABLE> by file name, join to <TLM_ITEM_TABLE> by load/batch id or source message id, join to <TLM_MATCH_RESULT_TABLE> and <TLM_EXCEPTION_TABLE> by item/recon keys, and check <TLM_WORKFLOW_QUEUE_TABLE> for queued/running/stuck states. Replace placeholders from the client schema.

Common follow-up

What happens after load?

Interviewer trap

Saying 'after load matching always happens'. It depends on initiation and proposal triggering.

What NOT to say

Do not invent physical table names.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

2. End-to-end reconciliation flow

Stage | What happens | Stored/logical data | Trigger | Failure | Diagnosis

Source system | Ledger, core banking, custody, SWIFT gateway, market data or internal sub-ledger produces data. | Source extracts/messages; source control totals; source transaction ids. | Scheduled extract/job/message event. | Extract not produced, upstream outage, wrong business date. | Check source job status, upstream confirmation, control totals.

Source file/message | Delimited/fixed/XML/SWIFT message carries transactions, balances or positions. | Raw file metadata, header/trailer counts, checksums, business date. | File delivery or message publication. | Malformed file, zero byte, wrong delimiter, wrong SWIFT tag/version. | head/tail/wc, parser logs, header/trailer count.

Tectia/SFTP/transfer | Secure transfer into bank/TLM integration area. | Transfer logs, timestamp, owner, permissions. | SFTP completion/rename convention/scheduler. | Partial file, permission denied, key expired, duplicate delivery. | ls -ltr, stat, checksum, transfer logs.

Landing/input directory | File waits for loader or integration route. | Raw inbound file and sometimes .done/.trigger marker. | Scheduler/ServiceMix/MI watches or batch picks file. | Wrong name/path, no read permission, stale file. | find latest file, permissions, user/group, disk space.

DMW/MI/mapping | Parse source, validate, transform, derive fields, perform lookups/defaults/rejects. | Load run, mapping version, rejects, target field values. | Loader execution or middleware route. | Bad datatype/date/amount, missing account, lookup miss. | MI/ETL logs, reject detail, row counts, compare source vs target.

TLM loading layer | Accept validated records into TLM logical business data areas. | Batch/load metadata, source/feed id, record counts. | Commit/successful load. | DB constraint, duplicate batch, partial commit, deadlock. | Load status, commit count, reject count, DB errors.

BDR/business data repository | Canonical business transaction/item/balance representation available for recon. | Items, balances, attributes, source lineage. | Item creation/enrichment/workflow initiation. | Wrong canonical value, no item created, wrong recon association. | Query by source transaction id/file id/business key.

Static configuration | Defines set, recon, data source/feed/account/rules/workflow/security. | Config metadata and effective-dated reference data. | Loader references static keys; recon enabled. | Invalid account/source/recon; inactive config. | Check active setup and dependencies.

Set/reconciliation setup | Groups related accounts/rules/model for a reconciliation objective. | Set code, recon code, class, data sources, accounts, workflows. | Item belongs to recon and workflow. | Wrong set or class leads to invisible/misrouted items. | Compare item keys to recon setup.

Data source/feed | Identifies side/source role used in matching, e.g., Ledger or Statement. | Feed id, side, source, mapping version. | Load creates side-specific items. | Both sides loaded as same source; missing side. | Count by data source/feed and side.

Entity/item creation | Create recon item/entity from business data for matching and exception lifecycle. | Item id, status, account, amount/qty, dates, source lineage. | Workflow initiation/proposal trigger. | No items, wrong status, duplicate items. | Trace source ref to item and status.

Enrichment | Derive normalized references, aliases, product/security mapping, customer/book/account attributes. | Derived fields, alias lookup results, enriched values. | Post-load enrichment or mapping rule. | Lookup miss, stale static, wrong derived key. | Compare raw vs enriched fields and lookup data.

Workflow initiation | Create work item/process state so matching/exceptions follow configured lifecycle. | Workflow state, queue entry, assigned queue/workfolder. | Item creation or schedule/manual action. | Not initiated, stuck queue, inactive process. | Workflow queue/status logs and DB logical area.

Scope | Select eligible data for matching for a pass/recon/run. | Eligibility criteria: recon, side, status, dates, accounts. | Matching run starts. | Too narrow=no candidates; too broad=bad performance/wrong groups. | Explain selected candidate count.

Population | Group potential candidates that could match. | Candidate groups by reference/account/currency/date/amount etc. | Scope output processed. | Good records not in same population; explosion of combinations. | Check grouping keys and candidate volume.

Matching/pass quality | Compare selected candidates and decide whether quality is sufficient. | Pass result, tolerance, exactness, matched attributes. | Population group evaluated. | No proposal, false proposal, wrong tolerance. | Review match pass diagnostics and rule order.

Proposal | System-suggested or pending/perfect match group before/at match creation depending config. | Proposal id/group, confidence/quality, status. | Pass quality satisfied. | Proposals generated but not accepted or not visible. | Query proposal/status and check UI filters/permissions.

Match | Final matched group; items status updated; possible automatic closure of exceptions. | Match result/group id, audit, item status. | Auto match/manual approval. | Wrong grouping, duplicate match, broken link. | Trace items to match group and audit.

Exception/break | Unmatched or quality-failed items represented as actionable issue. | Exception type, reason/resolution, inbox, SLA, ageing. | Matching leaves unresolved condition or manual raise. | Not raised, wrong type, wrong queue. | Check exception rules/workflow and dashboard filters.

TLM View/dashboard | Users investigate, filter, sort, assign, add notes/attachments, resolve. | Dashboard metadata, view filters, audit actions. | User opens dashboards or scheduled reports. | Missing records due to filters/access/data not loaded. | Reproduce with admin view; compare DB count to dashboard.

Audit/reporting | Full operational/control record of loading, matching, exception handling and approvals. | Audit history, comments, maker-checker, reports. | Every controlled action. | Incomplete audit/report mismatch. | Check audit trail by item/exception/load/user/time.

30-second interview explanation:

Data comes from source systems as files/messages, usually transferred by SFTP/Tectia or middleware. MI/DMW maps, validates and enriches it into TLM's business data area. Static setup connects the data to the correct set, account, source/feed, reconciliation class and workflow. Matching then uses scope, population and pass quality rules to create proposals/matches or exceptions. Operations users investigate breaks in TLM View, with audit and reporting around the full lifecycle.

3-minute senior explanation:

I never treat reconciliation as one step. I break it into connectivity, load, data quality, static association, workflow initiation, proposal triggering, matching, exception routing and dashboard visibility. For example, a ledger file may load successfully, but if the statement side is monthly and proposal triggering requires both inputs, then no matching should run yet. If data is missing from TLM, I trace from file arrival, load batch, reject count, item creation, reconciliation association, workflow queue, matching result and exception dashboard. At each layer I use a mix of file checks, logs, UI statuses and SQL against logical areas such as load metadata, item/business data, matching result, workflow queue, exception and audit. I avoid guessing physical table names until I verify the client schema.

3. Static setup

Object | Definition | Purpose | Where configured | Depends/depended | If wrong | Logical backend Set | Container/grouping for related reconciliation configuration and operational processing. | Groups accounts/recons/rules, provides operational boundary. | Recon Admin/SmartStudio depending deployment. | Reconciliation, accounts, workflow, dashboards. | Items routed to wrong population or not visible. | Configuration/static metadata area; VERIFY IN CLIENT SCHEMA.

Reconciliation | Specific recon process comparing defined data sources/sides for a business purpose. | Defines objective, sides, account eligibility, rules, workflow, triggering. | Recon Admin/SmartStudio. | Class, data sources, accounts, match passes, workflow. | No match, wrong match, wrong exception route. | Recon config metadata; VERIFY IN CLIENT SCHEMA.

Reconciliation Class | Template/model/category of recon behavior such as cash, position, securities, intersystem. | Defines semantics and available dimensions/rules/proofs. | Admin/model setup. | Reconciliation setup, allowed fields, matching/proof behavior. | Wrong behavior/dimensions/proofs. | Class/model metadata; VERIFY.

Data Source | Logical side/source role used in a reconciliation, e.g. Ledger, Statement, Custodian, Broker. | Separates inputs and side-specific matching logic. | Recon/static setup. | Feeds, items, match rules, views. | Both sides misclassified or missing side. | Source/data-source metadata; VERIFY.

Message Feed | Configured inbound stream/file/message definition from a source. | Controls parsing, mapping, schedule, lineage and target side. | MI/DMW/SmartStudio/admin/integration layer. | Load, mapping, data source, account mapping. | File loads to wrong side or rejects. | Feed/load metadata; VERIFY.

Account | Business account, nostro/depot/GL/custody/safekeeping unit being reconciled. | Primary control unit for matching/proofing/exceptions. | Static/reference/admin setup. | Items, balances, workflow, proofing. | Breaks misallocated, data invisible, proof wrong. | Account/reference metadata; VERIFY.

Business Data | Canonical fields used by TLM after mapping/enrichment. | Allows consistent matching/viewing independent of source format. | SmartSchema/MI mapping/model. | Items, matching, dashboards. | Wrong amount/date/ref impacts matching. | Business data/SmartSchema metadata and item data; VERIFY.

Source | Originating application/provider/counterparty/system. | Lineage and ownership of data. | Static/feed setup. | Feed, data source, items. | Tracing and duplicate control fail. | Source metadata; VERIFY.

Item | Reconcilable business record created from transaction/movement/balance data. | Unit of matching, status and exception lifecycle. | Created by load/workflow initiation. | Matching, exception, audit. | Missing item = cannot match/visible. | Business/item data area; VERIFY.

Asset | Security/instrument/cash asset dimension. | Critical for position/security matching and proofing. | Reference/static/security master integration. | Position balances, movements, matching. | Quantity breaks or wrong security. | Asset/reference metadata; VERIFY.

Entity | Business/legal/operational party depending client model. | Ownership, permissions, reporting and routing. | Static/user/org setup. | Accounts, workfolder, reports. | Wrong ownership and dashboard access. | Entity/reference metadata; VERIFY.

Business Unit | Operational/business ownership unit. | Routing, reporting, access, SLA. | Static/org setup. | Accounts, queues, users. | Break sent to wrong team. | Org/reference metadata; VERIFY.

Customer | Client/counterparty/customer dimension. | Client-level reporting and routing. | Static/reference setup. | Accounts, exceptions, reporting. | Wrong client investigation. | Customer/reference metadata; VERIFY.

Currency | ISO currency or currency-like dimension. | Cash amount matching/proofing and FX handling. | Static/reference setup. | Accounts, items, balances, tolerance. | False breaks or cross-currency errors. | Currency/reference metadata; VERIFY.

Group | Grouping of users/accounts/work/inboxes depending model. | Access and workload segmentation. | Security/workflow/admin setup. | Queues, dashboards, roles. | Users cannot see/own work. | Security/workflow metadata; VERIFY.

Workfolder | Operational container/list for work items/exceptions. | Work allocation and triage. | TLM Control/TLM View/admin. | Workflow queues/inboxes/users. | Breaks not accessible to team. | Workflow metadata; VERIFY.

Queue | Processing or operational queue for work/matching/exceptions. | Decouples process stages and work assignment. | Workflow/control/integration setup. | Workflow processors/services. | Stuck records, delayed matching/routing. | Queue logical area; VERIFY.

Workflow | Configured lifecycle steps and transitions. | Controls initiation, approvals, exception routing, actions. | TLM Control. | Items, exceptions, dashboards. | No initiation, stuck states, missing approvals. | Workflow/process metadata and audit; VERIFY.

Initiation | Rule/action that starts a workflow or match process for items/account/day. | Moves loaded data into processing lifecycle. | TLM Control/recon config/schedule/manual action. | Queue, matching, exception handling. | Loaded data remains idle. | Workflow initiation/run metadata; VERIFY.

Mandatory fields required to create static setup - interview-safe but not generic

Exact mandatory screen field names are version/client implementation dependent. The below are mandatory logical fields normally required to create a working setup; the exact labels must be verified in the client Recon Admin/SmartStudio/SmartSchema model.

Setup area | Mandatory logical fields | Why mandatory | Implementation-specific additions

Set/reconciliation | Unique set/recon code and name; active/effective status; reconciliation class/type; business date/calendar/time zone where used; data source/sides; eligible accounts/entities; match pass/rule set; workflow/initiation; owner/business unit; permissions/maker-checker approval. | Without these, the system cannot identify the recon, know what data belongs to it, run the correct matching model, or route the outcome. | Region, product, book, legal entity, SLA, priority, proof type, archival policy, regulatory tags.

Message feed | Unique feed/source id; source system; format/parser type; file naming or endpoint pattern;

schedule/trigger; record layout; field mapping to business data; target data source/side/recon; mandatory business keys; duplicate controls; reject handling; active status. | Without this, files cannot be parsed, mapped, traced or associated to a reconciliation. | Checksum markers, trailer count rules, encryption, compression, archive paths, ServiceMix route, batch window.

Account/static data | Account id; source account number/reference; internal account/GL/depot/safekeeping account; currency or asset eligibility; business unit/legal entity; customer/counterparty; active/effective dates; account type; reconciliation/set association. | Without account mapping, items cannot be controlled, proofed, routed or reported correctly. | Nostro/vostro indicator, BIC, branch, book, cost center, statement account alias, custodian, settlement location.

Position-specific | Depot/safekeeping account; asset/security identifier; quantity type; settled/unsettled bucket if used; opening/closing balance usage; movement eligibility; custody source. | Position recons rely on quantity and asset/location dimensions, not just amount. | ISIN/CUSIP/SEDOL priority, corporate action flags, pledge/loan buckets, location code.

Workflow/queue | Process/workflow name; initiation event; target queue/workfolder/inbox; roles/users/groups; statuses/transitions; SLA/ageing rules; exception type mapping. | Without workflow, loaded items or breaks may not be processed or visible to users. | Maker-checker levels, escalation email, manual match approvals, operations region routing.

Exact relationship - Set -> Source -> Feed -> Account -> Reconciliation -> Workflow:

- Set is the operational/configuration container.
- Source identifies where data originates, such as ledger, statement provider, custodian or broker.
- Feed is the concrete inbound stream from that source, including format, mapping and delivery controls.
- Account links inbound data to a business control unit such as nostro account, GL account, depot or safekeeping account.
- Reconciliation uses the class/model, accounts and data sources/sides to decide what is compared and how.
- Workflow defines what happens after item creation/matching/break creation: queueing, assignment, approval, investigation, resolution and closure.

Interview Question

What is Set in TLM static setup?

20-second answer

Set is container/grouping for related reconciliation configuration and operational processing. It supports groups accounts/recons/rules, provides operational boundary.

1-minute answer

In interview language, Set belongs to static/configuration rather than transaction data. It is configured through the administrative/model tools used by the client and downstream objects depend on it.

Senior 5+ year answer

At 5+ year level I explain the dependency and failure mode. If Set is wrong, the symptom is usually not just a database issue; it can lead to wrong routing, wrong matching, missing dashboard data or control/reporting gaps. Backend representation is a logical Configuration/static metadata area; VERIFY IN CLIENT SCHEMA.

Real project example

When a ledger record with reference ABC123 loads but appears under the wrong statement account, I would check Set setup only after confirming file/load/mapping lineage.

Technical deep dive

Depends on: Reconciliation, accounts, workflow, dashboards.. If wrong: Items routed to wrong population or not visible..

SQL/backend investigation

Query by active/effective configuration keys and compare item attributes: source, feed, account, recon, business date and status. Physical table names are implementation/version dependent.

Common follow-up

Which table changed when you created it?

Interviewer trap

Reciting unverified table names.

What NOT to say

Do not say transaction tables are updated merely because static setup was created. Static setup impacts config/reference metadata first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

What is Reconciliation in TLM static setup?

20-second answer

Reconciliation is specific recon process comparing defined data sources/sides for a business purpose. It supports defines objective, sides, account eligibility, rules, workflow, triggering.

1-minute answer

In interview language, Reconciliation belongs to static/configuration rather than transaction data. It is configured through the administrative/model tools used by the client and downstream objects depend on it.

Senior 5+ year answer

At 5+ year level I explain the dependency and failure mode. If Reconciliation is wrong, the symptom is usually not just a database issue; it can lead to wrong routing, wrong matching, missing dashboard data or control/reporting gaps. Backend representation is a logical Recon config metadata; VERIFY IN CLIENT SCHEMA.

Real project example

When a ledger record with reference ABC123 loads but appears under the wrong statement account, I would check Reconciliation setup only after confirming file/load/mapping lineage.

Technical deep dive

Depends on: Class, data sources, accounts, match passes, workflow.. If wrong: No match, wrong match, wrong exception route..

SQL/backend investigation

Query by active/effective configuration keys and compare item attributes: source, feed, account, recon, business date and status. Physical table names are implementation/version dependent.

Common follow-up

Which table changed when you created it?

Interviewer trap

Reciting unverified table names.

What NOT to say

Do not say transaction tables are updated merely because static setup was created. Static setup impacts config/reference metadata first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

What is Reconciliation Class in TLM static setup?

20-second answer

Reconciliation Class is template/model/category of recon behavior such as cash, position, securities, intersystem. It supports defines semantics and available dimensions/rules/proofs.

1-minute answer

In interview language, Reconciliation Class belongs to static/configuration rather than transaction data. It is configured through the administrative/model tools used by the client and downstream objects depend on it.

Senior 5+ year answer

At 5+ year level I explain the dependency and failure mode. If Reconciliation Class is wrong, the symptom is usually not just a database issue; it can lead to wrong routing, wrong matching, missing dashboard data or control/reporting gaps. Backend representation is a logical Class/model metadata; VERIFY.

Real project example

When a ledger record with reference ABC123 loads but appears under the wrong statement account, I would check Reconciliation Class setup only after confirming file/load/mapping lineage.

Technical deep dive

Depends on: Reconciliation setup, allowed fields, matching/proof behavior.. If wrong: Wrong behavior/dimensions/proofs..

SQL/backend investigation

Query by active/effective configuration keys and compare item attributes: source, feed, account, recon, business date and status. Physical table names are implementation/version dependent.

Common follow-up

Which table changed when you created it?

Interviewer trap

Reciting unverified table names.

What NOT to say

Do not say transaction tables are updated merely because static setup was created. Static setup impacts config/reference metadata first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

What is Data Source in TLM static setup?

20-second answer

Data Source is logical side/source role used in a reconciliation, e.g. ledger, statement, custodian, broker. It supports separates inputs and side-specific matching logic.

1-minute answer

In interview language, Data Source belongs to static/configuration rather than transaction data. It is configured through the administrative/model tools used by the client and downstream objects depend on it.

Senior 5+ year answer

At 5+ year level I explain the dependency and failure mode. If Data Source is wrong, the symptom is usually not just a database issue; it can lead to wrong routing, wrong matching, missing dashboard data or control/reporting gaps. Backend representation is a logical Source/data-source metadata; VERIFY.

Real project example

When a ledger record with reference ABC123 loads but appears under the wrong statement account, I would check Data Source setup only after confirming file/load/mapping lineage.

Technical deep dive

Depends on: Feeds, items, match rules, views.. If wrong: Both sides misclassified or missing side..

SQL/backend investigation

Query by active/effective configuration keys and compare item attributes: source, feed, account, recon, business date and status. Physical table names are implementation/version dependent.

Common follow-up

Which table changed when you created it?

Interviewer trap

Reciting unverified table names.

What NOT to say

Do not say transaction tables are updated merely because static setup was created. Static setup impacts config/reference metadata first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

What is Message Feed in TLM static setup?

20-second answer

Message Feed is configured inbound stream/file/message definition from a source. It supports controls parsing, mapping, schedule, lineage and target side.

1-minute answer

In interview language, Message Feed belongs to static/configuration rather than transaction data. It is configured through the administrative/model tools used by the client and downstream objects depend on it.

Senior 5+ year answer

At 5+ year level I explain the dependency and failure mode. If Message Feed is wrong, the symptom is usually not just a database issue; it can lead to wrong routing, wrong matching, missing dashboard data or control/reporting gaps. Backend representation is a logical Feed/load metadata; VERIFY.

Real project example

When a ledger record with reference ABC123 loads but appears under the wrong statement account, I would check Message Feed setup only after confirming file/load/mapping lineage.

Technical deep dive

Depends on: Load, mapping, data source, account mapping.. If wrong: File loads to wrong side or rejects..

SQL/backend investigation

Query by active/effective configuration keys and compare item attributes: source, feed, account, recon, business date and status. Physical table names are implementation/version dependent.

Common follow-up

Which table changed when you created it?

Interviewer trap

Reciting unverified table names.

What NOT to say

Do not say transaction tables are updated merely because static setup was created. Static setup impacts config/reference metadata first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

What is Account in TLM static setup?

20-second answer

Account is business account, nostro/depot/gl/custody/safekeeping unit being reconciled. It supports primary control unit for matching/proofing/exceptions.

1-minute answer

In interview language, Account belongs to static/configuration rather than transaction data. It is configured through the administrative/model tools used by the client and downstream objects depend on it.

Senior 5+ year answer

At 5+ year level I explain the dependency and failure mode. If Account is wrong, the symptom is usually not just a database issue; it can lead to wrong routing, wrong matching, missing dashboard data or control/reporting gaps. Backend representation is a logical Account/reference metadata; VERIFY.

Real project example

When a ledger record with reference ABC123 loads but appears under the wrong statement account, I would check Account setup only after confirming file/load/mapping lineage.

Technical deep dive

Depends on: Items, balances, workflow, proofing.. If wrong: Breaks misallocated, data invisible, proof wrong..

SQL/backend investigation

Query by active/effective configuration keys and compare item attributes: source, feed, account, recon, business date and status. Physical table names are implementation/version dependent.

Common follow-up

Which table changed when you created it?

Interviewer trap

Reciting unverified table names.

What NOT to say

Do not say transaction tables are updated merely because static setup was created. Static setup impacts config/reference metadata first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

What is Business Data in TLM static setup?

20-second answer

Business Data is canonical fields used by tlm after mapping/enrichment. It supports allows consistent matching/viewing independent of source format.

1-minute answer

In interview language, Business Data belongs to static/configuration rather than transaction data. It is configured through the administrative/model tools used by the client and downstream objects depend on it.

Senior 5+ year answer

At 5+ year level I explain the dependency and failure mode. If Business Data is wrong, the symptom is usually not just a database issue; it can lead to wrong routing, wrong matching, missing dashboard data or control/reporting gaps. Backend representation is a logical Business data/SmartSchema metadata and item data; VERIFY.

Real project example

When a ledger record with reference ABC123 loads but appears under the wrong statement account, I would check Business Data setup only after confirming file/load/mapping lineage.

Technical deep dive

Depends on: Items, matching, dashboards.. If wrong: Wrong amount/date/ref impacts matching..

SQL/backend investigation

Query by active/effective configuration keys and compare item attributes: source, feed, account, recon, business date and status. Physical table names are implementation/version dependent.

Common follow-up

Which table changed when you created it?

Interviewer trap

Reciting unverified table names.

What NOT to say

Do not say transaction tables are updated merely because static setup was created. Static setup impacts config/reference metadata first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

What is Source in TLM static setup?

20-second answer

Source is originating application/provider/counterparty/system. It supports lineage and ownership of data.

1-minute answer

In interview language, Source belongs to static/configuration rather than transaction data. It is configured through the administrative/model tools used by the client and downstream objects depend on it.

Senior 5+ year answer

At 5+ year level I explain the dependency and failure mode. If Source is wrong, the symptom is usually not just a database issue; it can lead to wrong routing, wrong matching, missing dashboard data or control/reporting gaps. Backend representation is a logical Source metadata; VERIFY.

Real project example

When a ledger record with reference ABC123 loads but appears under the wrong statement account, I would check Source setup only after confirming file/load/mapping lineage.

Technical deep dive

Depends on: Feed, data source, items.. If wrong: Tracing and duplicate control fail..

SQL/backend investigation

Query by active/effective configuration keys and compare item attributes: source, feed, account, recon, business date and status. Physical table names are implementation/version dependent.

Common follow-up

Which table changed when you created it?

Interviewer trap

Reciting unverified table names.

What NOT to say

Do not say transaction tables are updated merely because static setup was created. Static setup impacts config/reference metadata first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

4. Reconciliation Class vs Data Source

Aspect | Reconciliation Class | Data Source

Definition | A model/template/category of reconciliation behavior and dimensions. | A logical input side/origin role participating in a reconciliation.

Responsibility | Defines what kind of recon it is: cash, position, securities, intersystem, balance/proof etc. | Identifies where a record came from and which side it belongs to: Ledger, Statement, Custodian, Broker.

Hierarchy | Higher-level model used by one or many reconciliations. | Configured within/for a reconciliation and linked to feeds/items.

Why required | Gives semantic model and available dimensions/rules/proofs. | Keeps multiple inputs separate so matching can compare correct sides.

Cash example | Cash reconciliation class supports amount, currency, account, value date, references, balances. | Ledger data source vs Bank Statement data source.

Position example | Position class supports asset/security, quantity, depot, settled/unsettled, balance proof. | Internal Holdings data source vs Custodian Statement data source.

Common trap | Calling it the source system. | Calling it the recon type/class.

Interview Question

What is the difference between Reconciliation Class and Data Source?

20-second answer

Reconciliation Class says what type/model of reconciliation it is; Data Source says which input side or origin the data belongs to.

1-minute answer

A cash reconciliation class defines cash-style dimensions such as amount, currency, account, date and proofing behavior. Data sources under it can be Ledger and Statement. Both are required because the class defines behavior, while data sources separate the records being compared.

Senior 5+ year answer

At senior level, I avoid mixing model and input. If I configure a position recon, the reconciliation class must support asset/quantity/depot/balance concepts. Then I define data sources like internal holding and custodian statement. Match rules compare fields across those data sources. If data is loaded into the wrong data source, even the correct class will not match correctly.

Real project example

Cash: class = Cash Recon; data sources = GL Ledger and MT940 Bank Statement. Position: class = Position Recon; data sources = Internal Holdings and MT535 Custodian Holdings.

Technical deep dive

The class is conceptual/model level; the data source is input/side level. Physical storage is client schema dependent.

SQL/backend investigation

Count items by recon class/recon and data source/side; if both sides show same data source or one side is zero, check feed mapping/static association.

Common follow-up

Why not just use source system? Because a single source system can supply multiple feeds/sides; a data source is the recon-side abstraction.

Interviewer trap

The trap is saying Data Source and Reconciliation Class are synonyms.

What NOT to say

Do not answer only with examples; explain hierarchy and responsibility.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

20-second answer: Reconciliation Class defines the recon model; Data Source identifies the

side/source of records.

1-minute answer: A cash class gives cash dimensions and proof behavior; data sources such as Ledger and Statement provide the actual sides.

Senior answer: The class controls semantics and available dimensions; data sources control lineage and side separation. Wrong class means wrong recon behavior. Wrong data source means correct records are loaded but compared/routed incorrectly.

5. Data loading

Confirmed public information: TLM Message Interface/MI is described as a dedicated data loader with UI-based loading configuration, enrichment, transformation and lookup capabilities. Public material says it supports CSV/delimited, fixed-length, Excel, XML and SWIFT ISO 15022/ISO 20022 message types, with validations and error/reload handling. The same public material also describes integrated ETL, scheduled active data extraction and acceptance of mapped data from industry-standard tools.

Interview principle: do not equate file arrival, DB load, item creation, workflow initiation and matching. They are different controls.

Question | Senior answer

Which table does loaded data first enter? | Do not invent a physical table. Conceptually the first durable write may be load/batch control, staging, validation/reject or business data depending architecture. Answer with chain: source -> ingestion/load -> validation/mapping -> BDR/item -> recon association -> workflow -> matching. Verify client schema.

How identify whether file actually loaded? | Check file arrival, loader/job status, load batch metadata, accepted/rejected counts, commit time, file id, and item count by source transaction id/file id.

How identify rejected data? | Check reject files/tables/logical reject area, validation error messages, mapping logs, missing mandatory fields, data type/date/amount errors, lookup misses.

How identify partial loading? | Compare source trailer/header count vs accepted + rejected; compare file line count excluding headers/trailers; check commits and DB error logs.

How identify duplicate loading? | Check duplicate file name/business date/source batch id; count duplicate business keys; compare existing load ids; look for idempotency/duplicate status.

How identify feed generated an item? | Use source lineage fields: file id, batch id, feed id/source id, source message id, record number, mapping version.

How trace one transaction? | Search source file by reference, confirm record number, map to load batch, map to item by source transaction id/record id, then to recon/workflow/match/exception/audit.

Interview Question

If data is loaded into DB, which tables are impacted?

20-second answer

Data loading impacts load metadata and business/item data areas; it may also create rejects, workflow initiation records and audit entries. Exact physical tables are client/version dependent.

1-minute answer

I would separate feed/load control from business item storage. A successful file usually updates batch/load status, accepted/rejected counts, source/feed lineage, business transaction/item areas and audit. If workflow initiation is configured, workflow/queue areas may also get entries after item creation.

Senior 5+ year answer

At senior level, I say the first write could be staging/load control or direct business data depending architecture and persistence model. I will not name a table until I check the client's schema. I prove load using file id, batch id, accepted/rejected counts, item count and source-to-target reconciliation.

Real project example

For a ledger cash file, I compare trailer count 10,000 with load status accepted 9,980 and rejected 20, then inspect the reject reason - for example invalid account or invalid date format.

Technical deep dive

Physical table names are implementation/version dependent; the following represents the logical table/data area: load control, staging/reject, business data/item, workflow queue, audit.

SQL/backend investigation

```
SELECT load_id, file_name, status, accepted_count, rejected_count FROM <TLM_LOAD_BATCH_TABLE>  
WHERE file_name = :file_name;
```

Common follow-up

What table is first?

Interviewer trap

Giving a fake table name.

What NOT to say

Do not say the item table is always first; staging/load metadata may be first.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

6. Backend logical tables

Logical area | Purpose | Example identifiers | What to query | How to validate

Static/configuration data | Sets, recons, classes, data sources, accounts, rules, workflow setup | set_code, recon_code, class_code, account_id, source_id | Active/effective config by code | Static changes do not normally create transaction items

Source/feed metadata | Source system, feed, parser/mapping, schedule/format | feed_id, source_id, format, mapping_version | Feed linked to file/load and target recon side | Wrong feed maps item to wrong data source

Batch/load metadata | File/job control, counts, status, timestamps | load_id, file_name, business_date, status, accepted/rejected_count | Accepted + rejected = source count | Partial/failed/duplicate loads

Business/item data | Canonical transaction/movement/item attributes | item_id, source_txn_id, account, amount, currency, date, ref | Source record traceable to item | Missing item or wrong field values

Reconciliation status | Item/account/day status in reconciliation lifecycle | recon_id, item_id, status, business_date | Loaded item associated to recon | Loaded but not visible/ineligible

Matching/proposal | Candidate groups, proposals, match groups, pass results | proposal_id, match_id, pass_name, quality_status | Matched/unmatched count and pass diagnostics | Matching not started or bad match

Workflow | Process state, work items, transitions | workflow_id, item_id/exception_id, state, queue | Initiated and transitioned | Stuck workflow

Exception | Break lifecycle, reason/resolution, owner/inbox/SLA | exception_id, type, reason, status, owner, age | Open/closed lifecycle correct | Break not raised/assigned

Audit/history | Who did what, when, before/after | entity_id, action, user, timestamp | Full audit trail for controlled events | Missing control evidence

Balance | Opening/closing, proof, position/cash balances | account, asset, currency, opening, closing, movement | Proof equation holds | Balance mismatch despite transaction match

Queue | Queued processing work for workflow/matching/integration | queue_id, object_id, status, created_time, attempts | No excessive age/retries | Service down or poison record

Security/user | Users, roles, groups, permissions | user_id, role, group, access | User can see correct dashboards/inboxes | Records exist but user cannot see them

Dashboard/view metadata | TLM View layouts, filters, columns, shared views | dashboard_id, widget, filter, role | UI criteria matches data | False missing data due filters/access

MMQ/workflow queue note: where MMQ/workflow queue terms are used, treat them first as logical queued work areas. In some client conversations MMQ may refer to a specific message/matching/middleware queue implementation. Do not invent the expansion or table name. Ask for/client documentation and then explain the same processing principle: a queue entry represents work waiting for a processor/service to perform the next lifecycle step.

7. MMQ / workflow queue / matching service

Implementation-specific terminology warning: Public SmartStream material confirms TLM Control workflow and TLM Matching, but publicly available material does not confirm one universal physical MMQ table name, workflow_queue table name or proprietary queue command. In interviews, answer using the logical queue concept and state that physical names are verified from the client's schema/runbook.

DATA LOAD -> item creation -> workflow initiation -> queue -> workflow/matching processor -> proposal -> match result -> exception/result visibility.

Question | Senior answer

What is MMQ? | Client/version dependent term; usually discussed as a queue/message mechanism used to decouple processing steps such as matching or workflow. I verify the exact expansion in client documentation.

What is workflow_queue? | A logical queued work area holding items/process events awaiting a workflow or matching processor. Physical name is client dependent.

Who polls workflow_queue? | A configured workflow processor, matching service, integration service or scheduler depending architecture. Verify service name in runbook.

Who triggers matching after load? | Item creation/workflow initiation plus proposal-triggering configuration and scheduler/service availability. Load alone is not enough.

Does data load automatically mean matching happens? | No. Matching depends on recon setup, triggering mode, required inputs, workflow initiation, queue processing and service availability.

What happens between load and matching? | Validation, item creation, recon association, enrichment, workflow initiation, queue entry, trigger evaluation.

How know matching started? | Check matching run/proposal/match result logical area, queue state transition, service logs and dashboard status.

Queue stuck? | Check oldest queue age, status/retry/error, service liveness, poison record, DB locks, failed dependencies; reprocess only with approval/runbook.

Matching service down? | Loads may complete and items may queue, but no proposals/matches are generated; service logs show down or no polling.

Data-load vs matching failure? | Load failure has reject/load errors and no/partial items. Matching failure has items loaded but no run/proposal/match or stuck queue/service errors.

Interview Question

Who is responsible for triggering matching after data load?

20-second answer

Not the file load alone. Matching is triggered by configured initiation/proposal rules, workflow/queue processing and the matching service/scheduler.

1-minute answer

After data loads, TLM must create items, associate them to the reconciliation, initiate workflow and evaluate proposal triggering. If the trigger condition is satisfied, a queue/run is picked by the matching processor and results are created.

Senior 5+ year answer

At senior level I would prove each transition. Load success: batch count and items. Initiation: workflow state/queue entry. Trigger: required sides/date schedule satisfied. Service: matching processor alive and logs show it picked work. Result: proposal/match/exception counts updated. If no matching, I do not reload the file blindly; I isolate whether initiation, trigger, queue or service failed.

Real project example

Ledger feed daily, statement monthly: daily ledger loads create items but may remain pending or unmatched until statement availability/monthly trigger. That is expected if proposal triggering requires both inputs or a date-driven month-end run.

Technical deep dive

Check logs: loader log, workflow/control log, matching log, integration/ServiceMix logs where applicable, DB errors. Check statuses: load status, workflow queue age, matching run status, exception/result dashboard.

SQL/backend investigation

```
SELECT status, COUNT(*), MIN(created_ts), MAX(updated_ts) FROM <TLM_WORKFLOW_QUEUE_TABLE> WHERE  
recon_id=:recon GROUP BY status;
```

Common follow-up

What if queue is stuck?

Interviewer trap

Restarting services without checking poison record/locks/retries and business impact.

What NOT to say

Do not say 'matching engine automatically polls everything' without checking client configuration.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

8. Matching

Match concept | Interview explanation

Exact matching | All configured fields match exactly, e.g., amount=amount, ref=ref, currency=currency.

One-to-one | One ledger item matches one statement item.

One-to-many | One statement amount equals several ledger items, e.g., batch payment.

Many-to-one | Several statement lines equal one ledger posting.

Many-to-many | Multiple records on both sides net or aggregate to same value.

Amount matching | Compare amounts; may require sign/debit-credit normalization.

Date matching | Compare value/trade/settlement dates; may allow tolerance windows.

Reference matching | Compare transaction reference, bank ref, narrative-derived ref or normalized reference.

Tolerance matching | Allow configured difference by amount, percentage, date days or FX converted amount.

Currency matching | Require same currency or approved FX/cross-currency rule.

Account matching | Require same mapped account/nostro/depot/GL relationship.

Asset matching | Require same security identifier/asset for position/security recon.

Composite matching | Use multiple fields together: account+currency+amount+value date+reference.

Partial matching | Propose or hold incomplete groups where one side or item is missing.

Netting | Aggregate debits/credits/quantities to net value before comparison.

Write-off | Controlled resolution for small differences within business approval policy.

Manual matching | User manually links items, often requiring permissions/tolerance/approval.

Interview Question

Explain one-to-many and tolerance matching in TLM.

20-second answer

One-to-many matches one item on one side to multiple items on the other; tolerance allows a controlled difference within configured business limits.

1-minute answer

Example: one bank statement credit of 1,000 may match four ledger credits of 250 each. If bank charges create a 2 dollar difference and policy allows it, tolerance matching can propose or auto-match depending pass quality and approval.

Senior 5+ year answer

At senior level I also check sign conventions, currency, account, value date and reference normalization. Tolerance is not a license to hide breaks; it must be approved, audited, and often restricted by user role or pass quality. Some tolerance differences should create write-off or investigation workflow, not silent auto-match.

Real project example

Cash recon: statement amount 998 vs ledger 1,000 due to fee. Rule groups by account, currency, reference and date, pass quality checks amount difference ≤ 2 and reason/type policy, then creates suggested match or write-off workflow.

Technical deep dive

Public SmartStream material confirms one-to-one, one-to-many, many-to-many, aggregate, tolerance and reference masking capabilities in TLM RP. Exact rule configuration screens/fields vary by version/client.

SQL/backend investigation

Compare candidate groups in <TLM_PROPOSAL_TABLE> to items in <TLM_ITEM_TABLE>; inspect amount difference, tolerance rule name, pass result and audit.

Common follow-up

Can tolerance auto-match?

Interviewer trap

Saying tolerance always auto-matches. It depends on pass quality, proposal type and approval controls.

What NOT to say

Do not set broad tolerances just to improve match rate.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

9. Scope / population / pass quality

Concept | Purpose | When executed | What it selects | What it compares | Example | Common mistake
 Scope | Limit eligible universe | At start of matching pass/run | Items by recon, status, account, date, side | Usually not deep comparison; selects candidates | Only unmatched USD items for account A and value date 20-Aug | Putting every match condition here
 Population | Group candidates that could belong together | After scope | Potential groups/candidates | Grouping keys may include ref/account/currency/date | Statement ABC123 and ledger ABC123 candidates | Population too broad causing huge combinations
 Match rule | Evaluate business equality/tolerance | Within population | Selected candidates | Amount/date/reference/currency/account/asset/etc. | Amount=1000, ref=ABC123, currency=USD | Confusing grouping with quality
 Pass quality | Decide if result is acceptable and categorize | After rule evaluation | Candidate group result | Quality criteria/thresholds/completeness | Perfect if all fields exact; suggested if date within 1 day | Ignoring quality and auto-matching weak candidates
 Proposal | Output group for match/manual review | After pass quality | Proposed matches or pending groups | N/A | Suggested match shown to user | Assuming proposal equals final match
 Workflow/queue | Route result/action | After proposal/match/break | Work item/exception | N/A | Break goes to cash investigations inbox | Not checking queue when nothing visible

Concrete cash example:

Statement: Amount 1000, Reference ABC123, Currency USD, Value date 20-Aug.

Ledger: Amount 1000, Reference ABC123, Currency USD, Value date 20-Aug.

Scope selects eligible unmatched items for the cash reconciliation, account, USD and open status. Population groups statement and ledger candidates with reference ABC123, same account/currency and nearby value date. Match rule compares exact amount, exact currency, exact reference and exact value date. Pass quality classifies the group as perfect because all mandatory attributes pass. Proposal/match creation then updates the items as matched and no break remains. If date was 21-Aug but date tolerance was allowed, pass quality may create a suggested rather than perfect match depending setup.

Why scope should not contain every matching condition: scope should control performance and eligibility. If you put all detailed match equality checks into scope, you lose diagnostics, pass hierarchy, tolerance handling and quality categorization. Pass quality is important because it distinguishes perfect auto-matches from suggested/manual/pending cases and preserves auditability.

10. Proposal triggering

Public SmartStream material confirms TLM RP can match immediately on arrival, scheduled at specific times, or manually requested. It also describes pending matches when a match group is incomplete and later data can complete it if rules pass. Exact proposal trigger names/options in a TLM 3.x implementation must be verified in the client configuration.

Trigger concept | Meaning | Use case | Risk/check

Any matching input | Run/propose when any configured side/feed arrives. | High-frequency transaction recon where late opposite side should still create pending/unmatched visibility. | May generate many one-sided breaks if expected side is late.

Both/all required inputs | Run only when required sides/feeds are available. | Cash/position control where comparing incomplete data would create false breaks. | If one side never arrives, no proposals/matches; monitor missing side.

Scheduled/date-driven | Run at defined time/business date/month-end after cut-off. | Monthly statement/position recon or end-of-day balance proof. | Wrong calendar/time zone/cut-off causes late or missing runs.

Manual/on-demand | User/support triggers matching after data correction/reload. | Rerun after reject fix or exception investigation. | Needs authorization and audit.

Feed availability | Trigger checks whether configured input batch/balance exists. | Position recon requiring ledger movements daily and custodian statement monthly. | Data load success is not the same as trigger satisfied.

Workflow initiation | Items must enter workflow/state eligible for matching. | Controlled environments with TLM Control lifecycle. | Loaded data sits idle if initiation fails.

Interview Question

Ledger feed arrives daily but statement feed arrives only on a specific day of the month. How would you configure proposal triggering for a position reconciliation?

20-second answer

I would not trigger final matching only on daily ledger arrival if the statement side is monthly.

I would use both-required or scheduled month-end triggering, with monitoring for missing statement/balance availability.

1-minute answer

For a position recon, internal ledger/holdings or movements can arrive daily, but custodian statement/MT535 may arrive month-end. I would configure feed availability and/or date-driven triggering so proposals run when the required statement/balance side is available, or at an agreed cut-off. Daily ledger data can load and be validated, but not necessarily produce final proposals until the external side arrives.

Senior 5+ year answer

At senior level I separate three things: data loading, workflow initiation and proposal triggering. Daily ledger loads should update load/item/balance areas and maybe workflow state. Proposal triggering should check business date, required sides, account/depot, asset/security balance availability and cut-off. If the statement is missing, the expected status may be pending/missing-side alert rather than a false break storm. I would add monitoring for missing monthly statement feed and proof/balance availability.

Real project example

Example: Internal holding movements load daily. On the 5th business day, custodian MT535 statement arrives. Trigger requires internal holding snapshot + custodian position balance for account/depot/security and business date. Matching compares asset/account/depot/settled quantity and creates matches or position breaks.

Technical deep dive

Do not confuse a scheduled file pickup with proposal triggering. A batch can load successfully and no matching run should start if required sides are unavailable.

SQL/backend investigation

Check <TLM_LOAD_BATCH_TABLE> for both feeds, <TLM_BALANCE_TABLE> for position balances, <TLM_WORKFLOW_QUEUE_TABLE> for queued match request, and <TLM_MATCH_RESULT_TABLE> for proposals/results. Placeholders only.

Common follow-up

What if business wants daily visibility?

Interviewer trap

Creating daily false breaks for missing monthly statement without business agreement.

What NOT to say

Do not say 'TLM will automatically know monthly statement frequency' without configured trigger/feed availability rules.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

11. Mirror vs double entry

Terminology warning: Mirror and Double Entry may be used differently across banks and TLM implementations. Treat the below as accounting/reconciliation concepts unless the client confirms exact TLM configuration behavior.

Aspect | Mirror | Double entry
 Meaning | A reflected/opposite-side representation created to compare or normalize a transaction across sides. | Accounting principle where each business event has debit and credit postings.
 Reconciliation use | Can help represent one source as an expected opposite entry or normalize signs. | Used when reconciling ledger postings where debit/credit legs must balance.
 Transaction representation | May create/expect a mirror image such as debit vs credit for same economic event. | Two accounting legs, e.g., debit cash, credit receivable/revenue.
 Debit/credit behavior | Often sign inversion/opposite side interpretation. | Debit/credit depends account type and accounting rules.
 Cash example | Bank statement credit may mirror internal ledger debit/credit depending account sign convention. | Payment posting has debit/credit GL legs; recon may focus on cash leg vs statement.
 Position example | Internal movement expected to mirror custodian movement direction after normalization. | Securities accounting event may have position and cash legs.
 Advantage | Simplifies comparison where sides use opposite signs. | Strong accounting control and balanced ledger proof.
 Disadvantage | Can confuse users if generated expectations are mistaken for real source records. | More complex; not every recon compares all accounting legs.
 Trap | Claiming mirror is a universal TLM feature with fixed behavior. | Saying double entry just means duplicate records.

Interview Question

What is the difference between Mirror and Double Entry?

20-second answer

Mirror is a reconciliation representation or sign/opposite-side concept; double entry is the accounting principle of debit and credit postings for one event.

1-minute answer

In cash recon, a bank statement credit may need to be compared with an internal ledger posting whose sign is opposite after account-normalization. That is a mirror-style reconciliation concept. Double entry means the bank's accounting event itself has debit and credit legs, and the ledger balances from an accounting perspective.

Senior 5+ year answer

At senior level I first clarify how the client uses the terms. I do not say TLM always implements mirror in one physical way. I explain that mirror helps normalize or generate expected counterparts for matching, while double entry is about accounting completeness. Reconciliation may compare only the cash leg to the statement, but production support must understand the accounting source so sign issues are not treated as breaks incorrectly.

Real project example

Example: Ledger stores debit as positive for nostro account, statement stores credit as positive. Mapping/enrichment normalizes sign so matching compares economic amount consistently. That is not the same as saying the ledger double-entry posting is duplicated in TLM.

Technical deep dive

Verify whether 'mirror' is configured as generated expected items, sign-normalization, paired item logic or just business terminology in that bank.

SQL/backend investigation

Check raw amount, debit/credit indicator, normalized amount, source side and match rule sign logic in logical item/business data area.

Common follow-up

Is mirror a TLM table?

Interviewer trap

Treating terminology as physical backend object.

What NOT to say

Do not call double entry duplicate loading.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

12. Cash reconciliation

General industry knowledge: Cash reconciliation compares internal cash ledger records against external bank/counterparty statements. Typical cash statement formats include SWIFT MT940/MT950 or ISO 20022 camt statements, plus bank-specific flat files. TLM-specific configuration uses cash-oriented dimensions such as account, currency, amount, value date, transaction reference, debit/credit indicator, opening/closing balance and proofing where configured.

Topic | Cash reconciliation

Sources | Internal ledger/GL/sub-ledger vs bank statement/nostro statement/correspondent bank feed.

Typical messages | MT940/MT950/camt.052/camt.053 or proprietary bank statement files.

Units | Monetary amount by currency.

Key fields | Account, currency, value date, posting date, amount, debit/credit, bank reference, ledger reference, narrative.

Balances | Opening balance, closing balance, movements, sometimes available/intraday balances.

Proof | Opening balance + movements = closing balance; ledger and statement balances compared.

Common breaks | Missing ledger, missing statement, amount mismatch, wrong value date, duplicate posting, charges/fees, FX, reversal, stale outstanding.

Investigation | Trace source file, account mapping, normalized sign, reference extraction, value date, amount, bank charges, exception owner.

Interview Question

Explain cash reconciliation in TLM.

20-second answer

Cash recon compares internal cash ledger postings with external bank statement transactions and balances to identify matched items and breaks.

1-minute answer

The common setup has Ledger and Statement data sources. Mapping normalizes account, currency, amount, sign, value date and reference. Matching can use exact or tolerant amount/date/reference rules. Unmatched or mismatched items become breaks routed to operations in TLM View.

Senior 5+ year answer

At senior level I also discuss proofing: opening balance plus movements should equal closing balance, and transaction matching alone does not guarantee the balance is correct. I check static account mapping, sign conventions, duplicate controls, value-date logic, pass quality and workflow routing.

Real project example

MT940 statement says USD account A has credit 1000 ref ABC123 on 20-Aug. Ledger has posting 1000 same ref/date/currency. Scope selects open USD account items, population groups by ref/account/currency/date, pass quality validates amount/date/ref and a perfect match is created.

Technical deep dive

General industry: MT940/MT950 are cash statement/account reporting messages; TLM-specific: actual parser/mapping and physical tables are client dependent.

SQL/backend investigation

Use placeholders: query item data by account/currency/value date/reference and join load/feed lineage to match/exception status.

Common follow-up

Why cash break occurs even if transaction matched?

Interviewer trap

Ignoring balance/proof controls.

What NOT to say

Do not overclaim exact SWIFT tag parsing unless verified in project.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

13. Position reconciliation

General industry knowledge: Position/securities reconciliation compares internal holdings/positions and movements against custodian/depot statements. ISO 15022 sources commonly include MT535 Statement of Holdings, MT536 Statement of Transactions, MT537 Statement of Pending Transactions and MT548 settlement status/advice in securities workflows. TLM-specific setup depends on the position reconciliation class/model, security master/asset mapping, account/depot mapping, balance buckets and proof rules.

Topic | Position reconciliation

Sources | Internal books/holdings vs custodian/depot/prime broker statements.

Typical messages | MT535 holdings, MT536 transactions, MT537 pending, MT548 status; also proprietary custodian files.

Units | Quantity/units/face amount rather than cash amount; may include market value/currency.

Key fields | Depot/safekeeping account, asset/security id, ISIN/CUSIP/SEDOL, quantity, settlement status, trade date, settlement date, movement type.

Balances | Opening position, closing position, settled/unsettled, available/pledged/loan buckets depending requirement.

Proof | Opening position + buys/transfers in - sells/transfers out +/- corporate actions = closing position.

Common breaks | Missing movement, wrong security mapping, pending settlement, corporate action, quantity mismatch, unsettled/settled bucket mismatch.

Investigation | Trace asset id mapping, depot account, movement vs balance, settlement status, corporate action events, custodian timing.

Interview Question

Explain position reconciliation in TLM.

20-second answer

Position recon compares internal holdings or security positions with external custodian/depot positions, usually by account, asset/security and quantity.

1-minute answer

Unlike cash recon, the key unit is quantity/face amount and asset identity. Typical inputs include internal holdings/movements and custodian statements such as MT535/MT536. Matching uses depot account, security identifier, settlement date/status and quantity. Balance/proof is central.

Senior 5+ year answer

At senior level I emphasize that transaction matching can be correct while the closing position still breaks. For example, a corporate action, transfer or unsettled movement may affect balance buckets. I check opening position, movements, buys, sells, transfers, corporate actions, settled/unsettled classification and asset mapping.

Real project example

Internal position: 10,000 shares of ISIN X in depot D. Custodian MT535 says 9,500 settled and 500 pending. If internal stores 10,000 settled, transaction movements may match but position proof breaks by settled bucket.

Technical deep dive

General industry message purposes are public ISO 15022 knowledge; exact TLM balance table/model is client dependent.

SQL/backend investigation

Query <TLM_BALANCE_TABLE> by account/depot/security/business_date/bucket, then movement items and match status. Placeholders only.

Common follow-up

What is the balance table?

Interviewer trap

Treating position like cash amount-only recon.

What NOT to say

Do not ignore asset alias/security master issues.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

14. Balance and position balance table

What is the balance table in position reconciliation?

Senior answer: I would not name a physical table without verifying the schema. Conceptually, the balance logical area stores opening and closing balances/positions by account/depot/security/date and often by bucket such as settled, unsettled, pledged, available or location depending client design. It is required because a position recon is not only movement-to-movement matching; it must prove that opening position plus movements equals closing position and that internal and external closing positions agree.

Balance element | Why it matters | Example

Opening position | Starting quantity for proof. | Account D, ISIN X, opening 9,000.

Movements | Buys, sells, transfers, corporate actions adjust position. | +1,000 buy, -500 sell.

Closing position | End quantity to compare against custodian. | Expected 9,500.

Asset/security id | Identifies instrument; alias errors cause false breaks. | ISIN vs CUSIP mapping.

Quantity | Primary numeric unit for position recon. | Units or face amount.

Settled/unsettled | Bucket classification can cause break with same total. | 10,000 total but split differs.

Depot/account | Location/control account dimension. | Custody account/depot D.

Currency/market value | Relevant for valuation/NAV but not always core position quantity. | Market value in USD.

Example: Opening 100 shares. Buy 50 settles today, sell 20 pending. Internal closing settled = 150; custodian settled = 130 and pending sell = 20. Transaction matching of buy/sell may look correct, but settled position breaks because settlement status/bucket differs.

15. MI / Message Integration

Confirmed concept: Public SmartStream material describes TLM Message Interface/Message Integration as a data loading/integration capability with UI-based loading configuration, enrichment, transformation and lookup capabilities. It supports multiple source structures and message formats in public descriptions. It can validate syntactical correctness and business completeness, with failed messages available for review/correction/reload depending setup.

MI syntax accuracy rule: Publicly accessible material does not provide one universal TLM 3.x MI syntax. Do not invent syntax. In a client interview, say: I can explain MI mapping structure and testing approach, but exact syntax/export format is version/client implementation dependent and I would verify it from the client's MI definition/export or vendor documentation.

Question | Strong answer

What is MI? | TLM Message Interface/Message Integration is used to load, map, validate, transform and enrich inbound data into TLM business data structures.

What does MI do? | Accepts files/messages/extracts, parses record formats, maps source fields to TLM fields, derives values, performs lookups, validates and records rejects/load status.

What is MI syntax? | Exact syntax is version/client dependent. I explain logical structure: source definition, record layout/parser, field mapping, transformation/lookup/default rules, validation/reject handling, target data source/recon association.

Can you create MI independently? | You need source format, target SmartSchema/business fields, static setup, validation rules, permissions and environment-specific deployment/test process.

How source maps to TLM fields? | Through source-to-target mapping: source account -> TLM account, amount/sign -> normalized amount, date string -> business/value date, reference/narrative -> normalized reference, asset id -> asset.

How transformations performed? | Formula/derivation, lookup, conditional mapping, datatype conversion, defaulting, null handling, reference cleansing, sign normalization.

Error handling? | Reject invalid records, log validation/mapping errors, preserve source lineage, allow correction/reload if configured.

How test MI mapping? | Unit test with sample valid/invalid files, compare source count to accepted/rejected, field-level reconciliation, negative cases, duplicate file, partial file and reload.

Troubleshoot MI? | Check parser error, mapping version, mandatory fields, lookup/static failures, datatype/date/amount conversion, reject details and target item values.

Conceptual MI mapping structure (not real proprietary syntax):

SOURCE_DEFINITION: CASH_LEDGER_DAILY

FORMAT: DELIMITED pipe, header=true, trailer_count=true

TARGET_DATA_SOURCE: LEDGER

FIELD_MAP:

src.account_no	-> business.account_id	using ACCOUNT_ALIAS_LOOKUP
src.ccy	-> business.currency	validate ISO currency
src.amount	-> business.amount	decimal conversion
src.dr_cr	-> business.normalized_sign	conditional sign rule
src.value_date	-> business.value_date	parse YYYYMMDD
src.reference	-> business.reference	trim/uppercase/remove spaces

VALIDATIONS:

account_id required; currency required; amount numeric; value_date valid; reference required

REJECT_HANDLING:

reject record with reason; retain source file, line number and raw value; allow controlled correction/reload

16. DMW / data mapping / ETL

Your honest positioning: My strongest hands-on exposure is SQL plus DMW/data mapping/ETL around TLM. I should not claim deep Informatica unless I used it. I can confidently explain source-to-target mapping, transformation rules, validations, reconciliation of source vs target counts and how those concepts transfer to Informatica.

DMW capability | Interview explanation
 Source-to-target mapping | Document each source field, target TLM business field, datatype, mandatory flag and transformation.
 Transformation | Convert source value into usable canonical value: sign, date, amount, reference, currency.
 Derivation | Create derived fields such as normalized reference, business date, transaction type.
 Lookup | Map source account/security/customer codes to TLM static/master data.
 Conditional transformation | If dr_cr='D' then amount negative; if transaction type='FEE' route to fee category.
 Datatype conversion | String to date/number, decimal precision, thousand separator, comma decimal.
 Date conversion | YYYYMMDD/DD-MON-YYYY/source timezone to target date.
 Amount conversion | Sign normalization, decimal places, absolute/original amount preservation.
 Default values | Default source/system/business date only when approved; never mask missing mandatory fields.
 Null handling | Reject mandatory nulls; default optional values; log missing values.
 Reject handling | Reject field/record with reason, line number, raw value and correction route.
 Validation | Mandatory fields, datatype, lookup existence, duplicate keys, control totals.
 Testing | Unit files, negative files, source-target comparison, regression after mapping changes.
 File-level validation | Header/trailer counts, checksum, zero-byte, duplicate file name/business date.

Realistic cash-feed DMW example:

Source file columns: file_date, account_no, ccy, dr_cr, amount, value_date, bank_ref, narrative.

Target business fields: source_id, feed_id, account_id, currency, normalized_amount, debit_credit, value_date, reference, transaction_type, source_file, source_line_no.

Rules:

- account_id = lookup(account_no, account_alias_static). Reject if no active account mapping.
- currency = ccy upper-case; reject if not valid/active currency.
- normalized_amount = amount * -1 for debit if target convention requires debits negative; preserve original_amount separately if configured.
- reference = upper(trim(bank_ref)); if null, derive from narrative using approved extraction logic; if still null, reject or route to low-quality matching depending requirement.
- value_date = parse source date; reject invalid date.
- duplicate key = source_id + account_no + value_date + reference + amount + line/business date, according to approved idempotency rule.

Test: source count 1000, accepted 995, rejected 5. Reconcile accepted target amount sum by account/currency against source after normalization.

Interview Question

Describe your DMW/data mapping experience honestly.

20-second answer

My hands-on ETL exposure is strongest in SQL and DMW/data mapping around TLM, not deep Informatica development.

1-minute answer

I have worked on source-to-target mapping, transformations, lookups, validations, date/amount conversions, null handling, rejects and source-to-target reconciliation. Informatica concepts are familiar to me, but I would not claim to be an expert unless I actually built production workflows in it.

Senior 5+ year answer

At senior level I position it as transferable ETL knowledge. Whether the tool is DMW, MI or Informatica, the core is understanding source format, target SmartSchema/TLM fields, static lookups, control totals, reject handling, duplicate prevention and testing. I can adapt to

Informatica because mapping, transformations, sessions and workflows map closely to what I have done in DMW/SQL.

Real project example

For a cash feed, I mapped account, currency, amount, sign, date and reference, validated mandatory fields, rejected lookup failures and compared source counts/amounts with TLM loaded items.

Technical deep dive

Be transparent. Interviewers value honesty more than fabricated tool names.

SQL/backend investigation

Use SQL to reconcile source staging vs loaded items by file, account, currency, accepted/rejected counts and amount totals.

Common follow-up

Have you used Informatica?

Interviewer trap

Claiming deep Informatica when the real project used DMW.

What NOT to say

Do not say 'I developed complex Informatica workflows' unless true.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

17. Informatica - honest but strong answers

Question | Answer

What is your Informatica experience? | I have not had deep hands-on Informatica ownership. My direct ETL work is DMW and SQL. I understand Informatica concepts - mappings, transformations, sessions and workflows - and I can translate my DMW experience into Informatica quickly.

What is a mapping? | A design that defines how source data flows to target fields through transformations.

What is transformation? | A step that changes, validates, derives or filters data before target loading.

Source qualifier? | A source extraction component, typically controlling columns/filter/query from relational sources.

Lookup? | Finds reference/static/master data to enrich or validate source records, e.g., account alias to TLM account.

Expression transformation? | Derives fields using functions/formulas, e.g., normalized amount or parsed date.

Filter? | Allows only records meeting a condition to pass.

Router? | Routes records into multiple output groups based on conditions.

Joiner? | Joins data from two pipelines/sources where source-level join is not enough.

Aggregator? | Groups records and calculates sums/counts, useful for control totals.

Workflow? | Orchestrates mappings/sessions/tasks and dependencies.

Session? | Runtime execution of a mapping with connection, source/target and performance settings.

How map to DMW? | DMW mapping = Informatica mapping; DMW transformation rules = expression/filter/router/lookup; DMW batch/run = session/workflow; reject handling/control totals are common concepts.

18. Tectia / SFTP

Tectia is an enterprise SSH/SFTP/SCP secure file transfer and remote access solution. In TLM environments it is commonly part of the upstream delivery path: source system -> secure transfer -> landing directory -> loader/middleware. It differs from plain FTP because SFTP/SCP run over encrypted SSH channels and support stronger authentication and audit controls.

Question | Strong answer

How does a file reach TLM? | Source generates file, Tectia/SFTP transfers to landing path, scheduler/middleware/MI picks it up, mapping validates and loads it.

Verify file arrival? | Check directory, timestamp, file size, naming convention, marker file/checksum, transfer logs.

Check permissions? | `ls -l`, `stat`; verify owner/group/read permission for TLM loader user.

Missing file? | Check source generation, transfer job, network/key/account, target directory, time window, disk space.

Move/archive processed file? | Use approved batch/archive process; manually `mv` only with change/incident approval.

Zero byte file? | Do not load; confirm upstream, mark/reject per runbook, prevent false zero records.

Wrong naming convention? | Loader may not pick it up; validate naming pattern and business date.

Duplicate file? | Check duplicate controls by file name/hash/source batch/business date; avoid duplicate item creation.

Checksum failure? | Treat as integrity issue; do not process until re-delivered or approved.

19. Linux/Unix commands for TLM support

```

Command | Use
ls | List files/directories.
ls -ltr <dir> | List oldest-to-newest; tail shows latest files.
find <dir> -type f -name '*.dat' -printf '%T@ %p\n' | sort -n | tail | Robust latest-file search by
modification time.
grep 'ABC123' <file> | Find transaction/reference in file.
tail -100 <log> | Read latest log lines.
tail -f <log> | Follow log live; use carefully.
head -20 <file> | Check header/first lines.
cat <file> | Print small file.
less <file> | Page through large file.
wc -l <file> | Count lines for file control.
du -sh <dir> | Directory size.
df -h | Filesystem free space.
ps -ef | grep <process> | Find process.
top | Live CPU/memory view.
chmod 640 <file> | Change permissions.
chown user:group <file> | Change owner/group; requires privilege.
mv <src> <dest> | Move/rename/archive file.
cp <src> <dest> | Copy file.
rm <file> | Delete file; avoid in production unless approved.
mkdir -p <dir> | Create directory.
touch <file> | Create/update timestamp.
stat <file> | Detailed file metadata.
sort | Sort data/log extracts.
uniq -c | Count duplicates after sorting.
awk -F' | ' '{print $3}' | Extract columns from delimited files.
sed | Stream edit/filter text.
gzip/gunzip | Compress/decompress files.
scp/sftp | Secure copy/interactive secure transfer if approved.
date | Confirm server time/timezone.
id <user> | Check user group memberships.

```

Find latest file examples:

```

ls -ltr /tlm/inbound/cash | tail
find /tlm/inbound/cash -type f -printf '%T@ %p\n' | sort -n | tail -1

```

The first is easy and common; the second is more robust for scripting because it sorts by modification timestamp explicitly. Paths are examples only.

20. ServiceMix

ServiceMix is an open source Enterprise Service Bus/integration container based around integration technologies such as Karaf, ActiveMQ, Camel, CXF and OSGi concepts. Around TLM it may be used to route files/messages, connect endpoints, transform/mediate payloads or invoke downstream loaders/services. Exact deployment and configuration filenames are client-specific; do not invent them.

Question | Answer

Why used around TLM? | To integrate upstream/downstream systems, route messages/files, call loaders/services and decouple systems.

What configs normally present? | Logical functions: service/bundle config, endpoint config, routing rules, datasource/DB connectivity, logging, queue/integration properties, environment variables.

What is CFG/config file? | A configuration file/property set for an integration service or route; exact names depend on deployment.

Troubleshoot ServiceMix? | Check service status, route/bundle state, logs, endpoint availability, queue backlog, DB connectivity, recent config changes.

Know feed integration running? | Check process alive, route active, logs showing polling/processing, files moving from inbound to processing/archive, queue metrics.

Where check logs? | Environment runbook-defined log directory or container log; do not assume universal path.

Restart safely? | Follow runbook/change approval, check dependencies and backlog, stop gracefully, confirm no file mid-processing, restart, validate route and counts.

Config changes in prod? | Require change control, backup, impact analysis, SIT/UAT evidence, rollback plan and post-change validation.

21. TLM View

Confirmed public concept: TLM View is SmartStream's browser-based interface for operational, administrative and management reporting dashboards, with role-based controls, filtering, sorting, grouping, styling/personal/shared views, analytics, audit trail, notes and attachments in public material.

Relationship: Smart Studio/Recon Admin/SmartSchema/model configuration defines what data and attributes exist; TLM View exposes operational dashboards over underlying reconciliation data, workflow, exceptions and audit; WebConnect may be part of web access/integration depending client deployment.

Task | Interview answer

Create dashboard | Start from requirement: audience, dataset, columns, filters, grouping, security roles, actions. Configure via TLM View/admin tools available to client.

Add column | Expose configured attribute/SmartSchema field in dashboard widget; verify data exists and role can see it.

Change header | Update dashboard/widget column label where client tooling allows; avoid changing underlying data model unless needed.

Add filtering | Add default filters such as recon, account, status, business date, queue, age; allow user search if permitted.

Add sorting | Sort by age, amount, priority, business date or SLA.

Expose match/break | Include item status, match id/proposal status, exception type/reason/resolution, owner, queue and ageing.

Configure access | Use roles/groups/inboxes; test as target user.

Troubleshoot missing records | Check dashboard filters, user role, data loaded, workflow status, recon association, cache/index if applicable, and compare admin vs user view.

22. SQL - TLM-focused Oracle templates

Placeholders used below:

<TLM_ITEM_TABLE>, <TLM_LOAD_BATCH_TABLE>, <TLM_MATCH_RESULT_TABLE>, <TLM_EXCEPTION_TABLE>, <TLM_WORKFLOW_QUEUE_TABLE>, <TLM_BALANCE_TABLE>, <TLM_STATIC_ACCOUNT_TABLE>. Replace from client's schema, SmartSchema metadata, data dictionary, runbook or verified SQL already used by the support team.

SQL concept | TLM use

COUNT | Counts rows; COUNT(column) ignores NULLs.

COUNT DISTINCT | Counts unique non-null values.

GROUP BY | Aggregates by dimensions such as file/account/currency.

HAVING | Filters groups after aggregation, e.g., duplicates.

JOIN | Combines related rows from tables.

LEFT JOIN | Keeps left rows even when right side missing; useful for unmatched/missing checks.

NOT EXISTS | Find rows with no matching related rows.

EXISTS | Find rows with at least one related row.

CASE | Conditional output/aggregation.

ROW_NUMBER | Pick latest/first row per partition.

RANK | Rank with ties.

SUM | Totals amounts/quantities.

NVL/COALESCE | Replace null values.

CTE | WITH clause to structure complex logic.

Subquery | Query nested inside another query for filtering/aggregation.

No | SQL question | Oracle template

```
1 | file-wise matched record count | SELECT b.<FILE_NAME_COLUMN>, COUNT(*) AS matched_count<br>FROM
<TLM_ITEM_TABLE> i<br>JOIN <TLM_LOAD_BATCH_TABLE> b ON b.<LOAD_ID_COLUMN> = i.<LOAD_ID_COLUMN><br>WHERE
i.<MATCH_STATUS_COLUMN> = 'MATCHED'<br>GROUP BY b.<FILE_NAME_COLUMN>;
2 | file-wise unmatched record count | SELECT b.<FILE_NAME_COLUMN>, COUNT(*) AS unmatched_count<br>FROM
<TLM_ITEM_TABLE> i JOIN <TLM_LOAD_BATCH_TABLE> b ON b.<LOAD_ID_COLUMN>=i.<LOAD_ID_COLUMN><br>WHERE
i.<MATCH_STATUS_COLUMN> <> 'MATCHED' OR i.<MATCH_STATUS_COLUMN> IS NULL<br>GROUP BY b.<FILE_NAME_COLUMN>;
3 | records by source | SELECT i.<SOURCE_ID_COLUMN>, COUNT(*) FROM <TLM_ITEM_TABLE> i GROUP BY
i.<SOURCE_ID_COLUMN>;
4 | latest loaded file | SELECT * FROM (SELECT b.* FROM <TLM_LOAD_BATCH_TABLE> b ORDER BY
b.<LOAD_END_TS_COLUMN> DESC) WHERE ROWNUM = 1;
5 | duplicate records by business key | SELECT <BUSINESS_KEY_COLUMNS>, COUNT(*) FROM <TLM_ITEM_TABLE>
GROUP BY <BUSINESS_KEY_COLUMNS> HAVING COUNT(*) > 1;
6 | duplicate transaction references | SELECT <REFERENCE_COLUMN>, COUNT(*) FROM <TLM_ITEM_TABLE> WHERE
<REFERENCE_COLUMN> IS NOT NULL GROUP BY <REFERENCE_COLUMN> HAVING COUNT(*) > 1;
7 | missing statement records vs ledger | SELECT l.* FROM <TLM_ITEM_TABLE> l WHERE
l.<DATA_SOURCE_COLUMN>='LEDGER' AND NOT EXISTS (SELECT 1 FROM <TLM_ITEM_TABLE> s WHERE
s.<DATA_SOURCE_COLUMN>='STATEMENT' AND s.<REFERENCE_COLUMN>=l.<REFERENCE_COLUMN> AND
s.<AMOUNT_COLUMN>=l.<AMOUNT_COLUMN>);
8 | matched percentage by file | SELECT b.<FILE_NAME_COLUMN>, ROUND(100*SUM(CASE WHEN
i.<MATCH_STATUS_COLUMN>='MATCHED' THEN 1 ELSE 0 END)/COUNT(*),2) AS matched_pct FROM <TLM_ITEM_TABLE> i
JOIN <TLM_LOAD_BATCH_TABLE> b ON b.<LOAD_ID_COLUMN>=i.<LOAD_ID_COLUMN> GROUP BY b.<FILE_NAME_COLUMN>;
9 | break percentage by date | SELECT i.<BUSINESS_DATE_COLUMN>, ROUND(100*SUM(CASE WHEN
i.<MATCH_STATUS_COLUMN><>'MATCHED' OR i.<MATCH_STATUS_COLUMN> IS NULL THEN 1 ELSE 0 END)/COUNT(*),2) AS
break_pct FROM <TLM_ITEM_TABLE> i GROUP BY i.<BUSINESS_DATE_COLUMN>;
10 | account-wise breaks | SELECT i.<ACCOUNT_COLUMN>, COUNT(*) AS break_count,
SUM(ABS(i.<AMOUNT_COLUMN>)) AS break_amount FROM <TLM_ITEM_TABLE> i WHERE
NVL(i.<MATCH_STATUS_COLUMN>,'X') <> 'MATCHED' GROUP BY i.<ACCOUNT_COLUMN> ORDER BY break_amount DESC;
11 | currency-wise breaks | SELECT i.<CURRENCY_COLUMN>, COUNT(*) FROM <TLM_ITEM_TABLE> i WHERE
NVL(i.<MATCH_STATUS_COLUMN>,'X') <> 'MATCHED' GROUP BY i.<CURRENCY_COLUMN>;
12 | amount mismatch candidates | SELECT l.<REFERENCE_COLUMN>, l.<AMOUNT_COLUMN> ledger_amt,
s.<AMOUNT_COLUMN> stmt_amt, s.<AMOUNT_COLUMN>-l.<AMOUNT_COLUMN> diff FROM <TLM_ITEM_TABLE> l JOIN
<TLM_ITEM_TABLE> s ON s.<REFERENCE_COLUMN>=l.<REFERENCE_COLUMN> WHERE l.<DATA_SOURCE_COLUMN>='LEDGER' AND
s.<DATA_SOURCE_COLUMN>='STATEMENT' AND NVL(l.<AMOUNT_COLUMN>,0) <> NVL(s.<AMOUNT_COLUMN>,0);
13 | records loaded today | SELECT COUNT(*) FROM <TLM_ITEM_TABLE> WHERE TRUNC(<CREATED_TS_COLUMN>) =
TRUNC(SYSDATE);
14 | records loaded yesterday | SELECT COUNT(*) FROM <TLM_ITEM_TABLE> WHERE TRUNC(<CREATED_TS_COLUMN>) =
TRUNC(SYSDATE)-1;
15 | records with NULL mandatory fields | SELECT COUNT(*) FROM <TLM_ITEM_TABLE> WHERE <ACCOUNT_COLUMN> IS
NULL OR <CURRENCY_COLUMN> IS NULL OR <AMOUNT_COLUMN> IS NULL OR <REFERENCE_COLUMN> IS NULL;
16 | records stuck in workflow | SELECT <STATUS_COLUMN>, COUNT(*), MIN(<CREATED_TS_COLUMN>) oldest FROM
<TLM_WORKFLOW_QUEUE_TABLE> GROUP BY <STATUS_COLUMN>;
17 | records not matched | SELECT * FROM <TLM_ITEM_TABLE> WHERE NVL(<MATCH_STATUS_COLUMN>,'UNMATCHED') <>
'MATCHED';
18 | latest transaction per account | SELECT * FROM (SELECT i.*, ROW_NUMBER() OVER (PARTITION BY
i.<ACCOUNT_COLUMN> ORDER BY i.<VALUE_DATE_COLUMN> DESC, i.<CREATED_TS_COLUMN> DESC) rn FROM
<TLM_ITEM_TABLE> i) WHERE rn=1;
19 | highest break amount | SELECT * FROM (SELECT i.* FROM <TLM_ITEM_TABLE> i WHERE
NVL(i.<MATCH_STATUS_COLUMN>,'X') <> 'MATCHED' ORDER BY ABS(i.<AMOUNT_COLUMN>) DESC) WHERE ROWNUM <= 10;
20 | ageing breaks | SELECT CASE WHEN SYSDATE-<CREATED_TS_COLUMN> < 1 THEN '0-1D' WHEN
SYSDATE-<CREATED_TS_COLUMN> < 3 THEN '1-3D' WHEN SYSDATE-<CREATED_TS_COLUMN> < 7 THEN '3-7D' ELSE '7D+'
END bucket, COUNT(*) FROM <TLM_EXCEPTION_TABLE> WHERE <STATUS_COLUMN> NOT IN ('CLOSED','CANCELLED') GROUP
BY CASE WHEN SYSDATE-<CREATED_TS_COLUMN> < 1 THEN '0-1D' WHEN SYSDATE-<CREATED_TS_COLUMN> < 3 THEN '1-3D'
WHEN SYSDATE-<CREATED_TS_COLUMN> < 7 THEN '3-7D' ELSE '7D+' END;
```

```

21 | balance difference | SELECT <ACCOUNT_COLUMN>, <ASSET_OR_CURRENCY_COLUMN>,
SUM(<INTERNAL_BALANCE_COLUMN>) internal_bal, SUM(<EXTERNAL_BALANCE_COLUMN>) external_bal,
SUM(<INTERNAL_BALANCE_COLUMN>)-SUM(<EXTERNAL_BALANCE_COLUMN>) diff FROM <TLM_BALANCE_TABLE> GROUP BY
<ACCOUNT_COLUMN>, <ASSET_OR_CURRENCY_COLUMN> HAVING SUM(<INTERNAL_BALANCE_COLUMN>) <>
SUM(<EXTERNAL_BALANCE_COLUMN>);
22 | file-wise reconciliation statistics | -- Oracle template for file-wise reconciliation
statistics<br>SELECT <DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM
<VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY
record_count DESC;
23 | date-wise statistics | -- Oracle template for date-wise statistics<br>SELECT <DIMENSION_COLUMNS>,
COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
24 | load accepted vs rejected | -- Oracle template for load accepted vs rejected<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
25 | find file by transaction reference | -- Oracle template for find file by transaction
reference<br>SELECT <DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM
<VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY
record_count DESC;
26 | items by feed and status | -- Oracle template for items by feed and status<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
27 | items loaded but no workflow | -- Oracle template for items loaded but no workflow<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
28 | workflow retry count | -- Oracle template for workflow retry count<br>SELECT <DIMENSION_COLUMNS>,
COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
29 | open exceptions by inbox | -- Oracle template for open exceptions by inbox<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
30 | exceptions by reason | -- Oracle template for exceptions by reason<br>SELECT <DIMENSION_COLUMNS>,
COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
31 | exceptions resolved today | -- Oracle template for exceptions resolved today<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
32 | manual matches today | -- Oracle template for manual matches today<br>SELECT <DIMENSION_COLUMNS>,
COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
33 | proposals pending approval | -- Oracle template for proposals pending approval<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
34 | proposal count by pass | -- Oracle template for proposal count by pass<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
35 | match count by rule | -- Oracle template for match count by rule<br>SELECT <DIMENSION_COLUMNS>,
COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
36 | duplicate file load | -- Oracle template for duplicate file load<br>SELECT <DIMENSION_COLUMNS>,
COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
37 | zero-record loaded files | -- Oracle template for zero-record loaded files<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
38 | partial files | -- Oracle template for partial files<br>SELECT <DIMENSION_COLUMNS>, COUNT(*) AS
record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
39 | late files | -- Oracle template for late files<br>SELECT <DIMENSION_COLUMNS>, COUNT(*) AS
record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
40 | source-total amount by file | -- Oracle template for source-total amount by file<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
41 | target-total amount by file | -- Oracle template for target-total amount by file<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
42 | account currency control | -- Oracle template for account currency control<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
43 | items with invalid account | -- Oracle template for items with invalid account<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
44 | items with invalid currency | -- Oracle template for items with invalid currency<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
45 | items with future value date | -- Oracle template for items with future value date<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
46 | date mismatch candidates | -- Oracle template for date mismatch candidates<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
47 | reference normalization issues | -- Oracle template for reference normalization issues<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
48 | position quantity mismatch | -- Oracle template for position quantity mismatch<br>SELECT

```

```

<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
49 | settled vs unsettled breaks | -- Oracle template for settled vs unsettled breaks<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
50 | cash opening closing proof | -- Oracle template for cash opening closing proof<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
51 | top 10 ageing exceptions | -- Oracle template for top 10 ageing exceptions<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
52 | records by business unit | -- Oracle template for records by business unit<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
53 | items changed by user | -- Oracle template for items changed by user<br>SELECT <DIMENSION_COLUMNS>,
COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE <BUSINESS_FILTERS><br>GROUP BY
<DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
54 | audit trail for transaction | -- Oracle template for audit trail for transaction<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
55 | files processed in last hour | -- Oracle template for files processed in last hour<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
56 | items awaiting statement side | -- Oracle template for items awaiting statement side<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;
57 | ledger-only aged breaks | -- Oracle template for ledger-only aged breaks<br>SELECT
<DIMENSION_COLUMNS>, COUNT(*) AS record_count<br>FROM <VERIFIED_TLM_LOGICAL_TABLE><br>WHERE
<BUSINESS_FILTERS><br>GROUP BY <DIMENSION_COLUMNS><br>ORDER BY record_count DESC;

```

Verbal SQL question: file-wise matched record count

Say this verbally:

I would join the verified TLM item/business table with the load/batch metadata table using load id or batch id, filter item status to matched, group by file name and count the rows. I will replace table and column placeholders after checking the client's schema.

Oracle template:

```

SELECT b.<FILE_NAME_COLUMN>, COUNT(*) AS matched_record_count
FROM <TLM_ITEM_TABLE> i
JOIN <TLM_LOAD_BATCH_TABLE> b
  ON b.<LOAD_ID_COLUMN> = i.<LOAD_ID_COLUMN>
WHERE i.<MATCH_STATUS_COLUMN> = 'MATCHED'
GROUP BY b.<FILE_NAME_COLUMN>
ORDER BY b.<FILE_NAME_COLUMN>;

```

SQL Server equivalent uses the same structure; date and null functions differ in other queries: NVL becomes ISNULL or COALESCE, TRUNC(date) becomes CAST(date AS date).

23. PL/SQL

For TLM interviews, PL/SQL is usually tested for batch validation, reconciliation reports, control totals, exception extracts and support utilities. Do not claim to update TLM production tables directly unless following approved vendor/client procedures. Production table updates require change approval, backup, impact analysis and audit.

PL/SQL topic | Interview relevance

Cursor | Iterate result sets for controlled validations/extracts, though set-based SQL is preferred for volume.

Procedure/package | Reusable support/report logic.

Exception handling | Capture and log errors without silent failure.

Bulk collect/forall | High-volume processing with performance control.

Commit/rollback | Critical in support scripts; avoid partial uncontrolled commits.

Autonomous transaction | Used cautiously for logging; know risks.

Scheduler/job | Batch validation or reporting jobs where approved.

Explain plan/index | Performance tuning for TLM investigation queries.

PL/SQL skeleton for a control-total validation report (placeholder only):

```
CREATE OR REPLACE PROCEDURE validate_tlm_file_load(p_file_name IN VARCHAR2) AS
    v_source_count NUMBER;
    v_loaded_count NUMBER;
BEGIN
    SELECT <TRAILER_COUNT_COLUMN> INTO v_source_count
    FROM <TLM_LOAD_BATCH_TABLE>
    WHERE <FILE_NAME_COLUMN> = p_file_name;

    SELECT COUNT(*) INTO v_loaded_count
    FROM <TLM_ITEM_TABLE> i
    JOIN <TLM_LOAD_BATCH_TABLE> b ON b.<LOAD_ID_COLUMN> = i.<LOAD_ID_COLUMN>
    WHERE b.<FILE_NAME_COLUMN> = p_file_name;

    IF v_source_count <> v_loaded_count THEN
        DBMS_OUTPUT.PUT_LINE('Count mismatch: source='||v_source_count||',
loaded='||v_loaded_count);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Counts matched');
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('File not found in verified load metadata');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: '||SQLERRM);
        RAISE;
END;
/
```

24. Production troubleshooting

Scenario | First check | Second check | Third check | SQL check | Log check | Resolution

File not received | Source job/transfer window | SFTP/Tectia logs | landing path/date | Query load metadata for expected file | Transfer/source logs | Ask upstream to resend or confirm no file; update business ETA

File received but not loaded | Name/path/permissions | loader schedule/service | parser config | No load batch row or failed status | MI/ETL/ServiceMix logs | Fix name/permission/service then controlled reprocess

File loaded but zero records | file size/header/trailer | parser recognized records | mapping filters | accepted_count=0 | loader parser logs | Reject empty/invalid file; obtain correct file

File partially loaded | accepted/rejected counts | reject reasons | commit status | accepted+rejected vs source count | reject/DB logs | Correct rejects/reload per runbook

Data loaded but not visible | dashboard filters | user permissions | recon association/workflow | item exists by file/ref but status/role mismatch | view/workflow logs | Fix filter/access/static/workflow

Wrong field mapping | source raw value | mapping version | target item value | compare source vs item fields | MI/DMW logs | Correct mapping, regression test, reload if approved

Wrong account | account alias/static | mapping lookup | effective dates | item account vs source account | mapping/static logs | Fix account mapping and reload/correct items per policy

Wrong source | feed config | data source side | source id mapping | count by source/feed | loader logs | Correct feed-to-data-source mapping

Wrong reconciliation | set/recon static | account eligibility | class/data source | item recon id/status | workflow/recon logs | Fix static association

Matching not triggered | load complete? | workflow initiated? | trigger requirements? | queue/run/result absent | matching/control logs | Satisfy trigger/restart service/requeue per runbook

Workflow queue stuck | queue age/status | processor service | DB locks/retries | oldest queue rows | workflow logs | Resolve poison record/service/lock; controlled reprocess

Matching service down | process/status | logs | queue backlog | items loaded but no proposals | matching logs | Restart under runbook and validate results

Items proposed but not matched | proposal status | approval/workflow | pass quality | proposal rows pending | matching/UI logs | Approve/manual action/tune rule

Wrong match rule | rule order | scope/population | pass quality | candidate groups and pass result | matching diagnostics | Tune rule; regression test

Duplicate matches | duplicate item load | rule too broad | manual action audit | duplicate business keys/match groups | matching audit | Unwind per controls; fix duplicate detection/rule

TLM View missing records | filters | permissions | data status | DB count vs UI count | view logs | Adjust filters/access/cache/index

Balance mismatch | opening balance | movements | closing balance | balance diff query | proof logs | Investigate missing/wrong movement or balance feed

Position mismatch | asset/depot/bucket | settlement status | corporate action | balance/movement by asset | position/proof logs | Correct mapping/settlement/corporate action handling

Cash mismatch | sign/value date/ref | charges/fees | duplicate/missing side | item comparison | matching logs | Investigate source/charge/reversal; tune mapping/rule

Production job failure | scheduler return code | logs | dependencies | job audit | batch/app logs | Rerun after fix with business approval

ServiceMix issue | route/service status | logs | endpoint/queue | integration load gap | ServiceMix logs | Restart/rollback config under change

Tectia issue | connection/key | transfer logs | remote file | no load row | SFTP/Tectia logs | Fix credentials/network; request resend

Database connection issue | DB listener/connectivity | app datasource | credentials/locks | DB error in logs | app/DB logs | DBA/app team resolution

Oracle error | error code | SQL/object | recent changes | failed SQL/logical area | DB/app logs | Fix data/constraint/performance issue with DBA

Permission issue | user/role/group | file ownership | dashboard access | user role query | security/audit logs | Grant/correct role via approved process

20 production support scenarios - spoken structure

1. File not received: What happened - user/batch reported file not received. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
2. File received but not loaded: What happened - user/batch reported file received but not loaded. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
3. File loaded but zero records: What happened - user/batch reported file loaded but zero records. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by

- file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
4. File partially loaded: What happened - user/batch reported file partially loaded. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 5. Data loaded but not visible: What happened - user/batch reported data loaded but not visible. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 6. Wrong field mapping: What happened - user/batch reported wrong field mapping. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 7. Wrong account: What happened - user/batch reported wrong account. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 8. Wrong source: What happened - user/batch reported wrong source. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 9. Wrong reconciliation: What happened - user/batch reported wrong reconciliation. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 10. Matching not triggered: What happened - user/batch reported matching not triggered. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 11. Workflow queue stuck: What happened - user/batch reported workflow queue stuck. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 12. Matching service down: What happened - user/batch reported matching service down. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 13. Items proposed but not matched: What happened - user/batch reported items proposed but not matched. First I confirm business impact, source/file/account/date. Next I check

- file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
14. Wrong match rule: What happened - user/batch reported wrong match rule. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 15. Duplicate matches: What happened - user/batch reported duplicate matches. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 16. TLM View missing records: What happened - user/batch reported tlm view missing records. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 17. Balance mismatch: What happened - user/batch reported balance mismatch. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 18. Position mismatch: What happened - user/batch reported position mismatch. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 19. Cash mismatch: What happened - user/batch reported cash mismatch. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.
 20. Production job failure: What happened - user/batch reported production job failure. First I confirm business impact, source/file/account/date. Next I check file/load/workflow/matching layer depending symptom. SQL - use verified logical area query by file/ref/account/date. Logs - source transfer, MI/DMW, workflow, matching, ServiceMix as applicable. Communication - give facts, affected accounts/counts, ETA and workaround. Fix - follow runbook/change approval. Prevention - monitoring, control totals, alerts, mapping regression or static governance.

25. Scenario questions

Interview Question

User says transaction ABC123 is missing from TLM

20-second answer

Trace source file, arrival, feed/load, mapping/reject, item creation, recon association, workflow, matching/result/exception.

1-minute answer

Trace source file, arrival, feed/load, mapping/reject, item creation, recon association, workflow, matching/result/exception. Then I prove it with SQL/logs/UI at each layer.

Senior 5+ year answer

At senior level I give a sequence, evidence, risk control and stakeholder communication. I avoid one-table answers.

Real project example

Cash recon example with ledger vs statement account and reference ABC123.

Technical deep dive

Layered diagnosis is more important than memorized table names.

SQL/backend investigation

Use logical placeholders such as <TLM_ITEM_TABLE>, <TLM_LOAD_BATCH_TABLE>, <TLM_WORKFLOW_QUEUE_TABLE>, <TLM_MATCH_RESULT_TABLE> and replace them from the client schema dictionary, SmartSchema metadata, vendor documentation or existing support runbook.

Common follow-up

How will you prove it from DB/logs/UI?

Interviewer trap

The trap is to recite random table names or say data load automatically means matching without checking proposal triggering and workflow initiation.

What NOT to say

Do not say exact table names, MI syntax or ServiceMix filenames unless you have verified them in that specific environment.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

Static created, which tables affected?

20-second answer

Configuration/reference metadata areas first, not transaction/match tables.

1-minute answer

Configuration/reference metadata areas first, not transaction/match tables. Then I prove it with SQL/logs/UI at each layer.

Senior 5+ year answer

At senior level I give a sequence, evidence, risk control and stakeholder communication. I avoid one-table answers.

Real project example

Cash recon example with ledger vs statement account and reference ABC123.

Technical deep dive

Layered diagnosis is more important than memorized table names.

SQL/backend investigation

Use logical placeholders such as <TLM_ITEM_TABLE>, <TLM_LOAD_BATCH_TABLE>, <TLM_WORKFLOW_QUEUE_TABLE>, <TLM_MATCH_RESULT_TABLE> and replace them from the client schema dictionary, SmartSchema metadata, vendor documentation or existing support runbook.

Common follow-up

How will you prove it from DB/logs/UI?

Interviewer trap

The trap is to recite random table names or say data load automatically means matching without checking proposal triggering and workflow initiation.

What NOT to say

Do not say exact table names, MI syntax or ServiceMix filenames unless you have verified them in that specific environment.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

Data loaded, which table affected first?

20-second answer

Architecture dependent; explain ingestion/load/staging/control then business/item chain.

1-minute answer

Architecture dependent; explain ingestion/load/staging/control then business/item chain. Then I prove it with SQL/logs/UI at each layer.

Senior 5+ year answer

At senior level I give a sequence, evidence, risk control and stakeholder communication. I avoid one-table answers.

Real project example

Cash recon example with ledger vs statement account and reference ABC123.

Technical deep dive

Layered diagnosis is more important than memorized table names.

SQL/backend investigation

Use logical placeholders such as <TLM_ITEM_TABLE>, <TLM_LOAD_BATCH_TABLE>, <TLM_WORKFLOW_QUEUE_TABLE>, <TLM_MATCH_RESULT_TABLE> and replace them from the client schema dictionary, SmartSchema metadata, vendor documentation or existing support runbook.

Common follow-up

How will you prove it from DB/logs/UI?

Interviewer trap

The trap is to recite random table names or say data load automatically means matching without checking proposal triggering and workflow initiation.

What NOT to say

Do not say exact table names, MI syntax or ServiceMix filenames unless you have verified them in that specific environment.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

What table do you fetch when error?

20-second answer

Identify layer first: file, load, mapping, BDR, workflow, matching, exception.

1-minute answer

Identify layer first: file, load, mapping, BDR, workflow, matching, exception. Then I prove it with SQL/logs/UI at each layer.

Senior 5+ year answer

At senior level I give a sequence, evidence, risk control and stakeholder communication. I avoid one-table answers.

Real project example

Cash recon example with ledger vs statement account and reference ABC123.

Technical deep dive

Layered diagnosis is more important than memorized table names.

SQL/backend investigation

Use logical placeholders such as <TLM_ITEM_TABLE>, <TLM_LOAD_BATCH_TABLE>, <TLM_WORKFLOW_QUEUE_TABLE>, <TLM_MATCH_RESULT_TABLE> and replace them from the client schema dictionary, SmartSchema metadata, vendor documentation or existing support runbook.

Common follow-up

How will you prove it from DB/logs/UI?

Interviewer trap

The trap is to recite random table names or say data load automatically means matching without checking proposal triggering and workflow initiation.

What NOT to say

Do not say exact table names, MI syntax or ServiceMix filenames unless you have verified them in that specific environment.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

Data flows from DB/source into TLM - what captures first?

20-second answer

File architecture: transfer/landing/loader. DB architecture: extractor/query/ETL/loader. DMW/MI maps before item creation.

1-minute answer

File architecture: transfer/landing/loader. DB architecture: extractor/query/ETL/loader. DMW/MI maps before item creation. Then I prove it with SQL/logs/UI at each layer.

Senior 5+ year answer

At senior level I give a sequence, evidence, risk control and stakeholder communication. I avoid one-table answers.

Real project example

Cash recon example with ledger vs statement account and reference ABC123.

Technical deep dive

Layered diagnosis is more important than memorized table names.

SQL/backend investigation

Use logical placeholders such as <TLM_ITEM_TABLE>, <TLM_LOAD_BATCH_TABLE>, <TLM_WORKFLOW_QUEUE_TABLE>, <TLM_MATCH_RESULT_TABLE> and replace them from the client schema dictionary, SmartSchema metadata, vendor documentation or existing support runbook.

Common follow-up

How will you prove it from DB/logs/UI?

Interviewer trap

The trap is to recite random table names or say data load automatically means matching without checking proposal triggering and workflow initiation.

What NOT to say

Do not say exact table names, MI syntax or ServiceMix filenames unless you have verified them in that specific environment.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Interview Question

Build new reconciliation from scratch

20-second answer

Requirement, source analysis, static, mapping, load, rules, trigger, workflow, view, testing, deployment, support.

1-minute answer

Requirement, source analysis, static, mapping, load, rules, trigger, workflow, view, testing, deployment, support. Then I prove it with SQL/logs/UI at each layer.

Senior 5+ year answer

At senior level I give a sequence, evidence, risk control and stakeholder communication. I avoid one-table answers.

Real project example

Cash recon example with ledger vs statement account and reference ABC123.

Technical deep dive

Layered diagnosis is more important than memorized table names.

SQL/backend investigation

Use logical placeholders such as <TLM_ITEM_TABLE>, <TLM_LOAD_BATCH_TABLE>, <TLM_WORKFLOW_QUEUE_TABLE>, <TLM_MATCH_RESULT_TABLE> and replace them from the client schema dictionary, SmartSchema metadata, vendor documentation or existing support runbook.

Common follow-up

How will you prove it from DB/logs/UI?

Interviewer trap

The trap is to recite random table names or say data load automatically means matching without checking proposal triggering and workflow initiation.

What NOT to say

Do not say exact table names, MI syntax or ServiceMix filenames unless you have verified them in that specific environment.

Confidence level

Medium-High; verify physical schema/configuration in the client environment.

Backend investigation map for missing transaction ABC123

Step | SQL idea | Logical area | Useful fields | Expected | Failure interpretation

1 Confirm source file | grep ABC123 source file; line count/header/trailer | Source file/landing | file name, line number, business date | Record exists | If absent, upstream issue
 2 Confirm file arrival | ls/stat/check transfer logs | File system/transfer | timestamp, size, owner | Complete non-zero file | Transfer/permission issue
 3 Confirm feed | find load by file/feed | Feed metadata | feed id, source id | Correct feed | Wrong route/name
 4 Confirm batch/load | query load status | Load metadata | load id, status, counts | Success or reject details | Load failed/partial
 5 Confirm mapping | compare raw vs target | Mapping/reject | source line, error, target field | Mapped values correct | Mapping/lookup issue
 6 Confirm item creation | query item by source ref/line/load | Item data | item id, account, amount | Item exists | Not created/rejected
 7 Confirm recon/set | item recon/account/source | Recon/static | set/recon/data source | Correct association | Static mapping wrong
 8 Confirm workflow | workflow item state | Workflow | state, initiated time | Initiated | Workflow failed
 9 Confirm queue | queue status/age | Queue | queue status/retries | Processed or pending | Queue stuck/service down
 10 Confirm matching | match/proposal by item | Matching | proposal/match id | Result exists | Trigger/rule issue
 11 Confirm result | item status/result | Result/status | matched/unmatched | Expected status | Mismatch
 12 Confirm exception | exception by item/ref | Exception | type, reason, owner | Break visible | Routing/view issue

Build a new cash reconciliation from scratch - 10 to 15 minute answer

1. Requirement gathering: objective, accounts, sources, frequency, SLA, users, controls
2. Source analysis: ledger file, bank statement file, fields, control totals
3. Target analysis: TLM business fields/SmartSchema/account model
4. Reconciliation objective: transaction match plus balance proof
5. Feed identification: ledger daily, statement daily/monthly, delivery routes
6. Static setup: set, recon, class, data sources, accounts, BU, currencies, workflow
7. Mapping: DMW/MI source-to-target, sign/date/ref normalization
8. Data load: sample files, accepted/rejected counts, lineage
9. Data validation: source vs target counts/totals/field values
10. Enrichment: account alias, reference normalization, transaction type
11. Reconciliation class: cash model
12. Data sources: Ledger and Statement
13. Match rules: exact pass, tolerant pass, one-to-many pass, fallback pass
14. Scope: open items by account/currency/business date/status
15. Population: group candidates by account/currency/reference/date
16. Pass quality: perfect vs suggested vs pending thresholds
17. Proposal triggering: immediate/scheduled/both sides depending feed timing
18. Workflow: queues, inboxes, SLA, maker-checker, approvals
19. Exception handling: types, reasons, resolutions, escalation
20. TLM View: dashboards for breaks, matched, ageing, proof
21. Unit testing: mappings/rules
22. SIT: end-to-end with upstream/downstream
23. UAT: business sign-off
24. Production deployment: config/code management and rollback
25. Post-production validation: counts, match rates, exceptions
26. Monitoring: feeds, loads, queues, match rates
27. Documentation: support runbook, design, test evidence

26. 100+ interview questions

No	Category	Question
1	A Introduction	Tell me about your TLM experience.
2	A Introduction	Which TLM modules have you used?
3	A Introduction	What was your role in implementation/support?
4	A Introduction	How do you avoid overstating tool experience?
5	B TLM architecture	Explain TLM architecture.
6	B TLM architecture	What is BDR?
7	B TLM architecture	What is SmartSchema?
8	B TLM architecture	What is TLM Control?
9	B TLM architecture	What is TLM View?
10	C Static setup	What is a Set?
11	C Static setup	What is a Source?
12	C Static setup	What is a Message Feed?
13	C Static setup	What is Account static?
14	C Static setup	Mandatory fields for static setup?
15	C Static setup	Set -> Source -> Feed -> Account -> Recon -> Workflow?
16	D Data loading	What happens when a file arrives?
17	D Data loading	How do you confirm load?
18	D Data loading	Partial load?
19	D Data loading	Duplicate load?
20	D Data loading	Which table first?
21	E DMW	Explain DMW mapping.
22	E DMW	Lookup failure?
23	E DMW	Amount conversion?
24	E DMW	Date conversion?
25	E DMW	Reject handling?
26	F MI	What is MI?
27	F MI	MI syntax?
28	F MI	How test MI?
29	F MI	How troubleshoot mapping?
30	G Reconciliation	What is cash recon?
31	G Reconciliation	What is position recon?
32	G Reconciliation	What is securities recon?
33	G Reconciliation	What is intersystem recon?
34	H Match rules	Exact matching?
35	H Match rules	One-to-many?
36	H Match rules	Many-to-many?
37	H Match rules	Tolerance matching?
38	H Match rules	Manual matching?
39	I Scope	What is scope?
40	I Scope	Why not put all conditions in scope?
41	I Scope	Bad scope symptoms?
42	J Population	What is population?
43	J Population	Population vs match rule?
44	J Population	What if population too broad?
45	K Pass Quality	What is pass quality?
46	K Pass Quality	Perfect vs suggested?
47	K Pass Quality	Why quality matters?
48	L Proposal triggering	Any input vs all input?
49	L Proposal triggering	Scheduled trigger?
50	L Proposal triggering	Monthly statement scenario?
51	L Proposal triggering	Data load vs proposal?
52	M Workflow	What is workflow initiation?
53	M Workflow	What is queue?
54	M Workflow	What is workfolder?
55	M Workflow	What is inbox?
56	N MMQ	What is MMQ?
57	N MMQ	Who polls queue?
58	N MMQ	Queue stuck?
59	N MMQ	Service down?
60	O Backend/database	Which tables static affects?
61	O Backend/database	Which tables data load affects?
62	O Backend/database	What table when error?
63	O Backend/database	Trace ABC123?
64	P SQL	File-wise matched count?
65	P SQL	Break percentage?
66	P SQL	Duplicate references?
67	P SQL	Ageing breaks?
68	P SQL	Balance diff?
69	Q PL/SQL	Procedure vs function?
70	Q PL/SQL	Exception handling?
71	Q PL/SQL	Bulk collect?
72	Q PL/SQL	Commit strategy?
73	R Linux	Find latest file?
74	R Linux	Check permissions?
75	R Linux	Find reference in file?
76	R Linux	Check disk?
77	S Tectia	What is Tectia?
78	S Tectia	SFTP vs FTP?
79	S Tectia	Missing file?
80	S Tectia	Duplicate file?

81		T ServiceMix		What is ServiceMix?
82		T ServiceMix		Why around TLM?
83		T ServiceMix		How troubleshoot routes?
84		T ServiceMix		Restart safely?
85		U TLM View		Create dashboard?
86		U TLM View		Add column?
87		U TLM View		Missing records?
88		U TLM View		Access issue?
89		V Cash		MT940?
90		V Cash		Opening/closing proof?
91		V Cash		Cash mismatch?
92		V Cash		Nostro?
93		W Position		MT535?
94		W Position		Position balance?
95		W Position		Settled vs unsettled?
96		W Position		Corporate action impact?
97		X Balance		What is balance table?
98		X Balance		Proof equation?
99		X Balance		Balance break with matched txns?
100		Y Exception		Exception lifecycle?
101		Y Exception		Reason/resolution?
102		Y Exception		Allocation?
103		Y Exception		Ageing?
104		Z Production support		File not loaded?
105		Z Production support		Matching not triggered?
106		Z Production support		Queue stuck?
107		Z Production support		Communicate to business?
108		Mixed senior follow-up		What do you tell business?
109		Mixed senior follow-up		How do you rollback?
110		Mixed senior follow-up		How do you test?
111		Mixed senior follow-up		How will you prove it?
112		Mixed senior follow-up		What happens next?
113		Mixed senior follow-up		Which logical table area?
114		Mixed senior follow-up		What log do you check?
115		Mixed senior follow-up		How avoid duplicate loading?
116		Mixed senior follow-up		How identify wrong static?
117		Mixed senior follow-up		How tune match rule?
118		Mixed senior follow-up		What do you tell business?
119		Mixed senior follow-up		How do you rollback?
120		Mixed senior follow-up		How do you test?
121		Mixed senior follow-up		How will you prove it?
122		Mixed senior follow-up		What happens next?
123		Mixed senior follow-up		Which logical table area?
124		Mixed senior follow-up		What log do you check?
125		Mixed senior follow-up		How avoid duplicate loading?
126		Mixed senior follow-up		How identify wrong static?
127		Mixed senior follow-up		How tune match rule?
128		Mixed senior follow-up		What do you tell business?
129		Mixed senior follow-up		How do you rollback?
130		Mixed senior follow-up		How do you test?

27. Mock interview

Use this section interactively. The interviewer should ask one question at a time, start easy, then challenge vague answers with follow-ups: which table/log, what happens next, how do you prove it, what can fail, and what would you communicate.

Question 1 (easy): Give me a 60-second overview of your SmartStream TLM experience and be clear about which tools you used hands-on.

Scoring after your answer: TLM knowledge /10, Database /10, Reconciliation /10, Production support /10, SQL /10, Communication /10, Senior-level confidence /10.

Follow-up bank:

1. You said data loaded. How do you prove it?
2. Which logical table area would you check first?
3. If file loaded but matching did not run, what next?
4. What is the difference between data source and reconciliation class?
5. Write verbal SQL for file-wise matched count.
6. What if queue is stuck?
7. How do you explain this to operations without blaming upstream prematurely?

28. Rapid-fire round

Term | One-line answer | Deeper answer

BDR | Business Data Repository/logical business data area holding canonical recon data. | It is the layer where mapped data becomes usable for matching, workflow, audit and reporting; physical implementation is client dependent.

Set | Container for related reconciliation setup. | Groups recons/accounts/rules/workflow boundaries.

Source | Originating system/provider. | Used for lineage and feed ownership.

Data Source | Logical input side in a recon. | Example Ledger vs Statement.

Reconciliation Class | Recon model/type. | Cash, position, securities etc.; controls semantics.

Message Feed | Inbound stream/file definition. | Format, mapping, schedule and target side.

MI | TLM Message Integration/Interface for loading/mapping. | Parses, validates, transforms, enriches data.

DMW | Data mapping/ETL workbench/process. | Source-to-target transformations and validations.

Scope | Eligibility selection. | Limits candidate universe for matching.

Population | Candidate grouping. | Groups items that could match.

Pass Quality | Quality threshold/categorization. | Decides perfect/suggested/pending/no match.

Proposal | System candidate match group. | May become match automatically or require user action.

Workflow | Lifecycle/routing process. | Controls queue, assignment, approvals and states.

MMQ | Client-specific queue term. | Verify exact meaning; conceptually queued work for processors.

Queue | Work waiting area. | Decouples processing stages.

Break | Unresolved mismatch/exception. | Actionable investigation item.

Match | Accepted matched group. | Items linked and status updated.

Mirror | Opposite/reflected representation or sign concept. | Client terminology dependent.

Double entry | Debit/credit accounting principle. | Not duplicate loading.

Cash recon | Ledger cash vs bank statement. | Amounts/currency/account/value date/ref/balances.

Position recon | Internal holdings vs custodian positions. | Asset/quantity/depot/settled buckets.

Balance | Opening/closing/proof quantity or amount. | Required for proofing beyond transaction match.

TLM View | Browser dashboard/UI. | Operational, admin and MI dashboards with role-based access.

Smart Studio | Configuration/modeling tool in TLM suite. | Used for solution configuration depending client version.

Recon Admin | Admin configuration area/tool. | Static/recon setup depending deployment.

TLM Control | Workflow/BPM tool. | Customizes process and routing without code where configured.

Tectia | Enterprise SSH/SFTP/SCP transfer. | Secure file transfer around feeds.

ServiceMix | Integration/ESB container. | Routes/messages/integration around TLM in some environments.

SIT | System Integration Testing. | End-to-end across systems.

UAT | User Acceptance Testing. | Business validates requirements and operations.

29. Final interview survival sheet

50 must-remember concepts

1. TLM RP is reconciliation-agnostic
2. TLM 3.x exact schema/config is client dependent
3. MI loads/maps/validates/enriches
4. DMW/ETL concepts transfer to Informatica
5. TLM Control handles workflow
6. TLM View is operational dashboard/UI
7. SmartSchema is metadata layer
8. Data load is not matching
9. Proposal triggering is separate
10. Workflow initiation matters
11. Scope selects eligible universe
12. Population groups candidates
13. Match rule compares
14. Pass quality categorizes
15. Proposal is not always final match
16. Perfect/suggested/pending are different
17. One-to-one/one-to-many/many-to-many
18. Tolerance requires governance
19. Manual matching needs audit
20. Cash recon focuses amount/currency/account/date/ref
21. Position recon focuses asset/quantity/depot/bucket
22. Balance proof can break despite txn match
23. Static setup affects config metadata
24. File load affects load/item metadata
25. Matching affects proposal/match/status
26. Exceptions affect exception/workflow/audit
27. Never invent table names
28. Use logical table areas
29. Source lineage is key
30. Batch/load id is key
31. Account mapping is critical
32. Data source is recon side
33. Recon class is model
34. Feed is concrete inbound stream
35. Set groups setup
36. Queue stuck is not file-load failure
37. Service down leaves backlog
38. Rejects show data quality/mapping failures
39. Partial load = accepted+rejected vs source count
40. Duplicate load requires idempotency
41. Tectia/SFTP transfer layer
42. ServiceMix/integration routes
43. Linux basics prove file state
44. SQL counts and joins prove facts
45. PL/SQL changes require controls
46. SIT validates end-to-end
47. UAT validates business use
48. Config/code management matters
49. Stakeholder communication matters
50. Honesty beats fabricated experience

30 must-remember backend investigation points

1. Start with layer identification
2. Confirm source produced record
3. Confirm file arrival
4. Confirm file is complete
5. Confirm feed picked file
6. Confirm load status
7. Confirm accepted/rejected counts
8. Confirm mapping version
9. Confirm reject reason
10. Confirm item created
11. Confirm source lineage
12. Confirm account mapping
13. Confirm data source side
14. Confirm recon association
15. Confirm workflow initiated
16. Confirm queue entry
17. Confirm queue not stuck
18. Confirm matching run
19. Confirm proposals
20. Confirm match result
21. Confirm exception
22. Confirm TLM View filters
23. Confirm user access
24. Confirm audit trail
25. Confirm balance/proof
26. Confirm duplicate status
27. Confirm partial load
28. Confirm service status
29. Confirm DB errors/locks
30. Document evidence and timeline

30 SQL patterns

1. Count by file
2. Count by status
3. Count distinct references
4. Group by account/currency
5. HAVING duplicates
6. Left join missing side
7. NOT EXISTS missing counterpart
8. EXISTS counterpart
9. CASE matched percentage
10. SUM amount totals
11. ABS break exposure
12. NVL/COALESCE status
13. ROW_NUMBER latest row
14. RANK top breaks
15. CTE source/target compare
16. Date truncation
17. Between business dates
18. Join load to item
19. Join item to match
20. Join item to exception
21. Queue age query
22. Open exceptions query
23. Balance difference
24. Accepted vs rejected

25. Trailer count compare
26. Duplicate file query
27. Null mandatory fields
28. Manual action audit
29. SQL Server CAST date
30. Oracle explain plan

20 Linux commands

1. ls
2. ls -ltr <dir>
3. find <dir> -type f -name '*.dat' -printf '%T@ %p\n' | sort -n | tail
4. grep 'ABC123' <file>
5. tail -100 <log>
6. tail -f <log>
7. head -20 <file>
8. cat <file>
9. less <file>
10. wc -l <file>
11. du -sh <dir>
12. df -h
13. ps -ef | grep <process>
14. top
15. chmod 640 <file>
16. chown user:group <file>
17. mv <src> <dest>
18. cp <src> <dest>
19. rm <file>
20. mkdir -p <dir>

20 TLM troubleshooting sequences

1. File not received: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
2. File received but not loaded: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
3. File loaded but zero records: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
4. File partially loaded: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
5. Data loaded but not visible: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
6. Wrong field mapping: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
7. Wrong account: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
8. Wrong source: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
9. Wrong reconciliation: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
10. Matching not triggered: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
11. Workflow queue stuck: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
12. Matching service down: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
13. Items proposed but not matched: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
14. Wrong match rule: file/load -> mapping -> item -> recon -> workflow -> matching ->

- exception -> view.
15. Duplicate matches: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
 16. TLM View missing records: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
 17. Balance mismatch: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
 18. Position mismatch: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
 19. Cash mismatch: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.
 20. Production job failure: file/load -> mapping -> item -> recon -> workflow -> matching -> exception -> view.

20 match-rule concepts

1. Exact matching
2. One-to-one
3. One-to-many
4. Many-to-one
5. Many-to-many
6. Amount matching
7. Date matching
8. Reference matching
9. Tolerance matching
10. Currency matching
11. Account matching
12. Asset matching
13. Composite matching
14. Partial matching
15. Netting
16. Write-off
17. Manual matching
18. Rule order
19. Candidate grouping
20. Match diagnostics

10 cash reconciliation scenarios

1. Perfect ledger-statement match
2. Amount mismatch
3. Value-date mismatch
4. Missing ledger
5. Missing statement
6. Duplicate statement line
7. Bank charge tolerance
8. FX/cross-currency issue
9. Reversal/chargeback
10. Opening/closing balance mismatch

10 position reconciliation scenarios

1. Perfect holdings match
2. Quantity mismatch
3. Wrong ISIN alias
4. Settled/unsettled bucket mismatch
5. Pending settlement
6. Corporate action adjustment

7. Transfer in/out missing
8. Depot account mismatch
9. Internal movement matched but balance breaks
10. Custodian statement late

10 production incidents

1. File not received
2. File received but not loaded
3. File loaded but zero records
4. File partially loaded
5. Data loaded but not visible
6. Wrong field mapping
7. Wrong account
8. Wrong source
9. Wrong reconciliation
10. Matching not triggered

10 bluff-test questions

1. Which exact table is updated first?
2. What is MI syntax?
3. What is MMQ table name?
4. Which ServiceMix cfg file did you edit?
5. Does load always trigger matching?
6. What table do you update to fix static?
7. Did you use Informatica hands-on?
8. Is mirror a TLM physical table?
9. Can you restart production without approval?
10. Can you improve match rate by increasing tolerance? Explain controls.